Computer Architecture
CS 622 / EE 520
Fall 2026

Lecture 20

Shahid Masud

Topics

- Virtual Memory
- Virtual Memory Operation
- TLB Translation Lookaside Buffer
- Page Fault and Page Table Size
- Caches and Virtual Memory
- Protection in Virtual Memory

Quiz Next Week

Further Readings

Reading Notes

Readings:
Appendix B of course textbook

Slide credits:
https://people.duke.edu/~tkb13/courses/ece250-2020su/slides/10-virtual-memory.pdf

Slides adapted from:

Virtual Memory Concept

Motivation (1)

- Computers these days run more than one program at once
- Problem:
  - Program A puts its variable X at address 0x1000
  - Program B puts its variable Q at address 0x1000
  - Conflict!
- Solution:
  - Allow each running program to have its own "view" of memory (e.g. "my address 0x1000 is different from your address 0x1000")

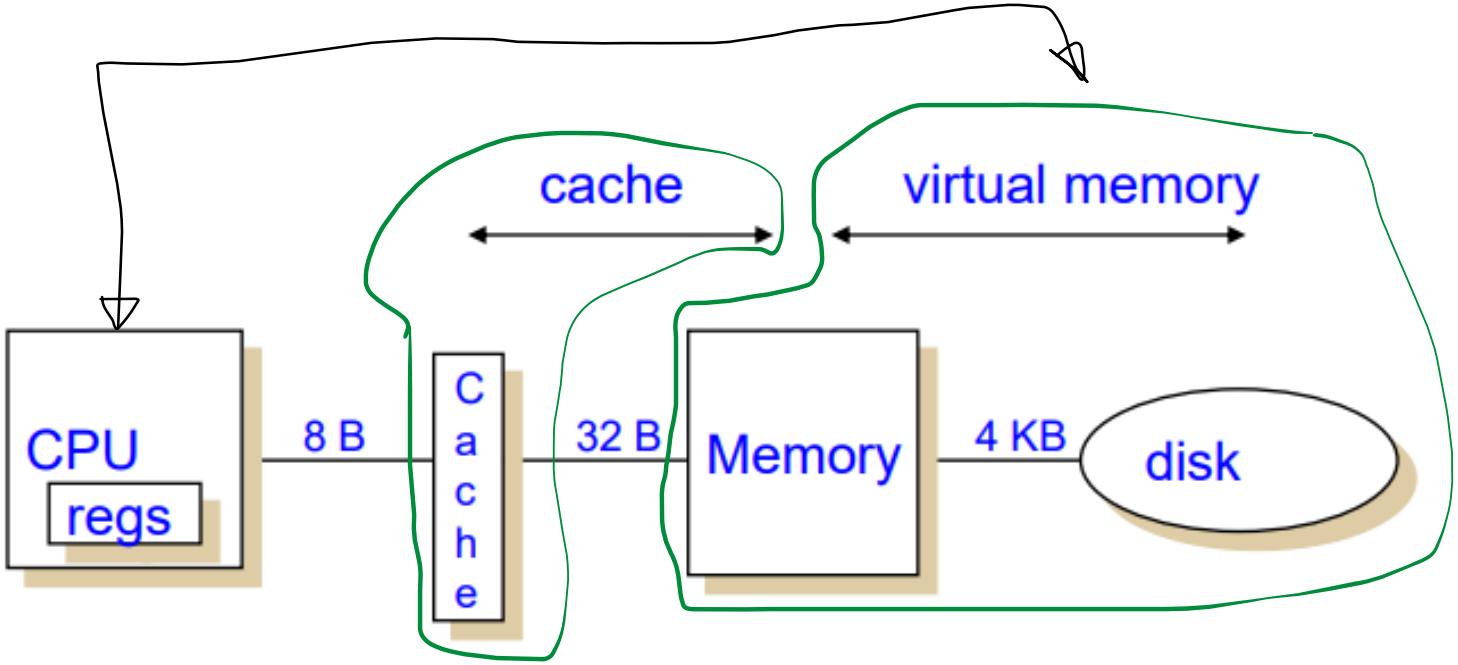
- How? Add a layer of **indirection** to memory addressing: **Virtual memory paging**

Motivation (2)

Motivations for Virtual Memory

- **Use Physical DRAM as a Cache for the Disk**
  - Address space of a process can exceed physical memory size
  - Sum of address spaces of multiple processes can exceed physical memory
- **Simplify Memory Management**
  - Multiple processes resident in main memory
    - Each process with its own address space
  - Only "active" code and data is actually in memory
    - Allocate more memory to process as needed
- **Provide Protection**
  - One process can't interfere with another
    - because they operate in different address spaces
  - User process cannot access privileged information
    - different sections of address spaces have different permissions

Levels in Memory Hierarchy



	Register	Cache	Memory	Disk Memory
size:	32 B	32 KB-4MB	1024 MB	100 GB
speed:	1 ns	2 ns	30 ns	8 ms
$/Mbyte:		$125/MB	$0.20/MB	$0.001/MB
line size:	8 B	32 B	4 KB	

larger, slower, cheaper →

Benefits of Virtual Memory

Paging benefits:

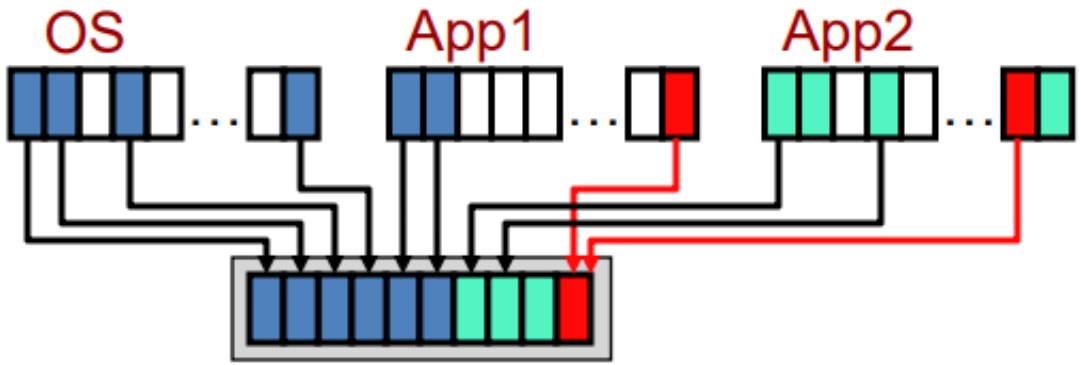
- **Simpler programs**:
  - We "virtualize" memory addresses, so every program believes it has a full 2^{32} or 2^{64} byte address space
- **Easier sharing**:
  - Processes can "share" memory, i.e. have virtual addresses that map to the same physical addresses

Swapping benefits:

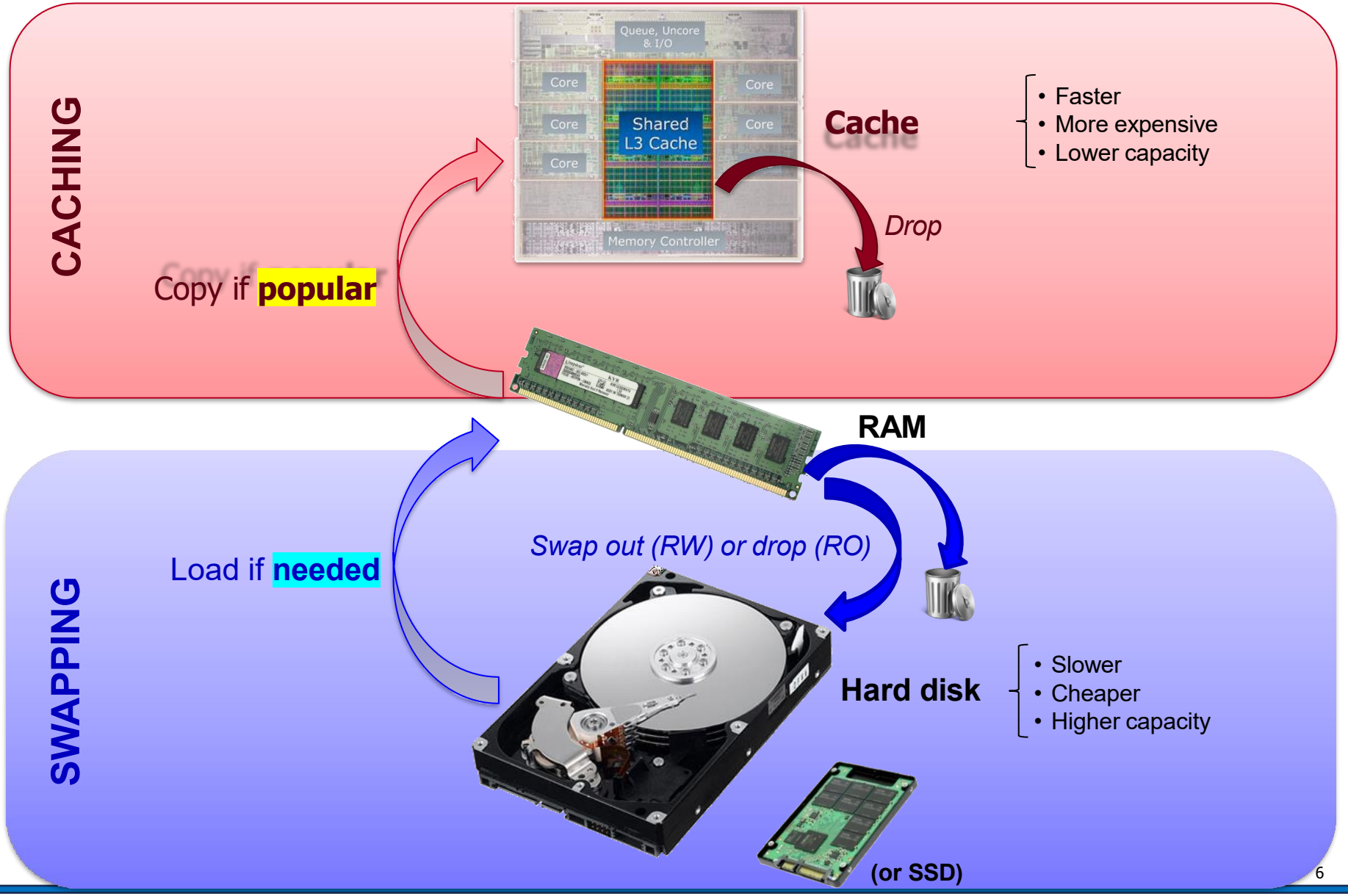
- **Bigger programs**:
  - Not just bigger than free memory, bigger than AVAILABLE memory!
- **More programs**:
  - Only part of each program loaded, so more can be loaded at once
- **Faster multitasking**:
  - Less effort to load or swap processes

Uses of Virtual Memory

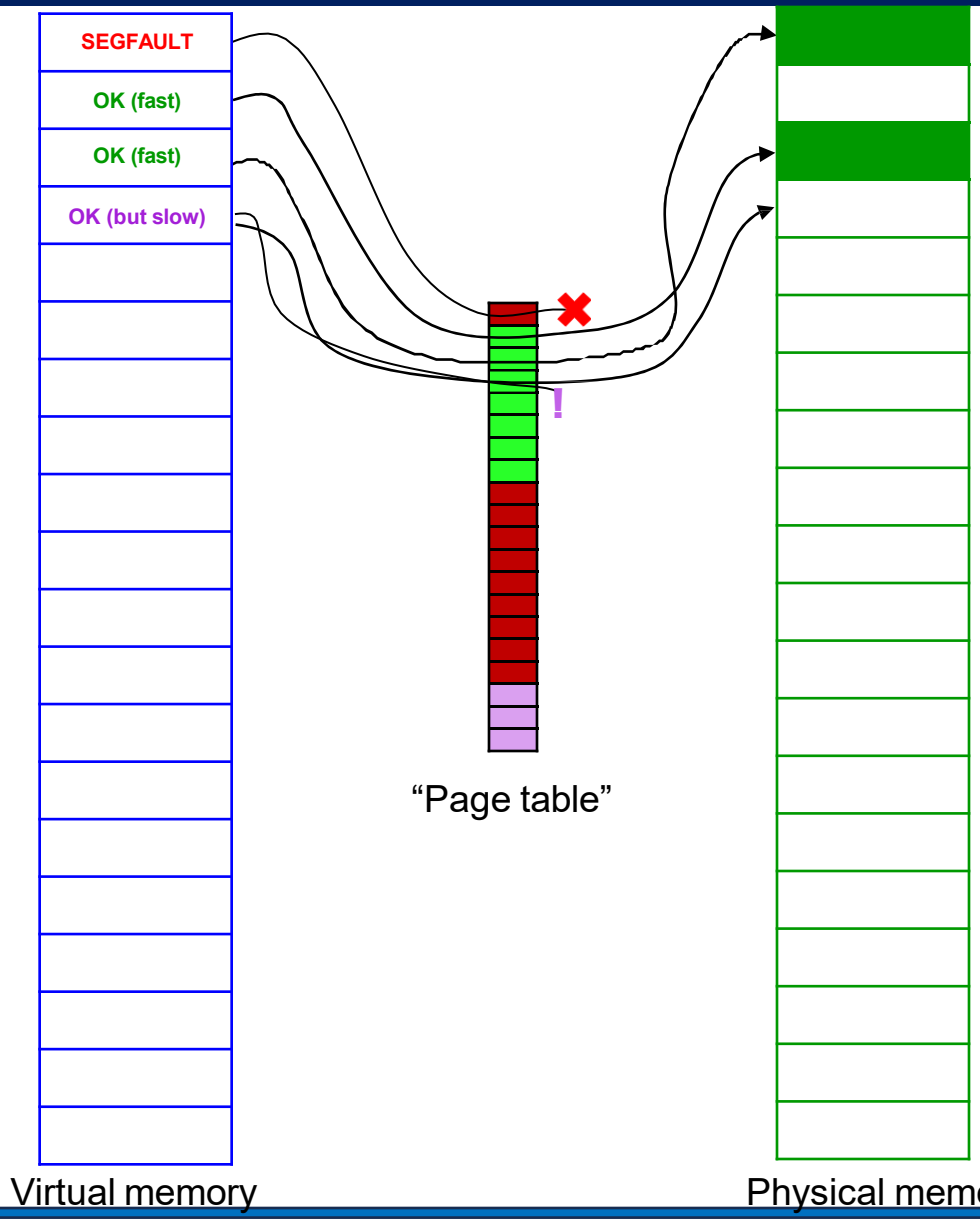
- **Isolation** and **Multi-programming (Memory Management)**
  - Each app thinks it has 2^N B of memory that starts @ 0
  - Apps can't read/write each other's memory
    - Can't even address the other program's memory!
- **Protection**
  - Each page has read/write/execute permission set by OS
  - Enforced by hardware
- **Inter-process communication**
  - Map same physical pages into multiple virtual address spaces
  - Or share files via the UNIX `mmap()` call

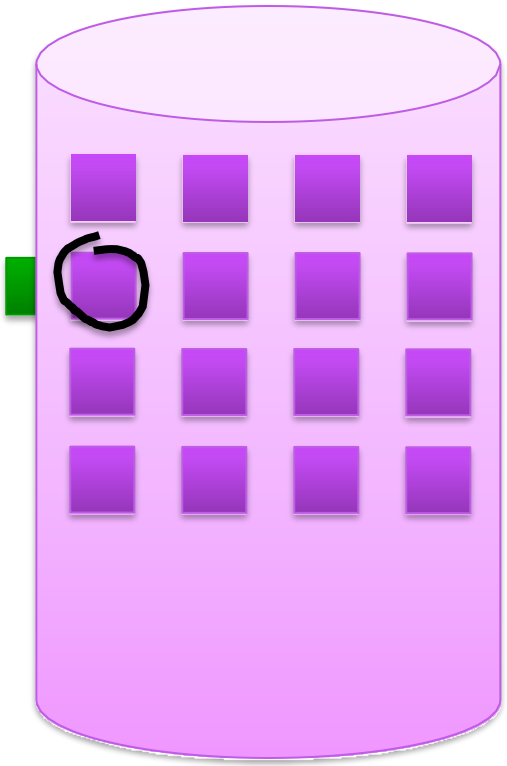


Caching and Swapping in Virtual Memory



High Level VM Operation Concept





Basic Terminology

Several general memory hierarchy ideas from Chapter 1 about caches are analogous to virtual memory, although many of the terms are different. *Page* or *segment* is used for block, and *page fault* or *address fault* is used for miss. With virtual memory, the processor produces *virtual addresses* that are translated by a combination of hardware and software to *physical addresses*, which access main memory. This process is called *memory mapping* or *address translation*. Today, the two memory hierarchy levels controlled by virtual memory are DRAMs and magnetic disks. Figure B.20 shows a typical range of memory hierarchy parameters for virtual memory.

Logical Program and Virtual Space

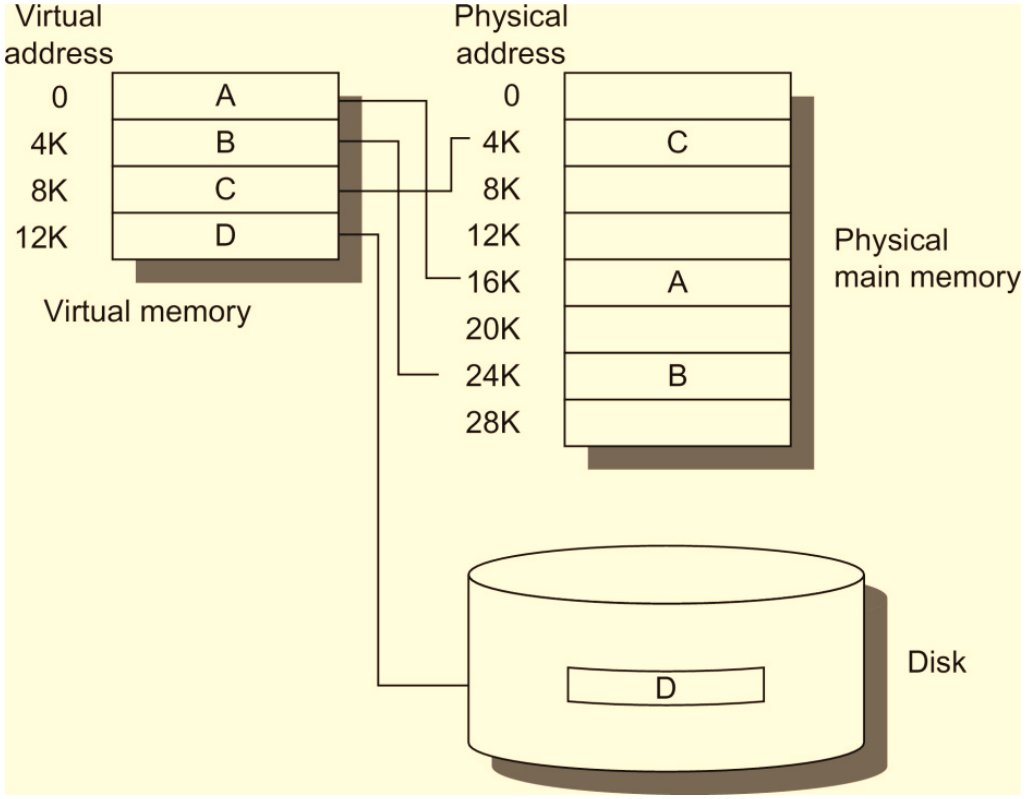


Figure B.19 The logical program in its contiguous virtual address space is shown on the left. It consists of four pages, A, B, C, and D. The actual location of three of the blocks is in physical main memory and the other is located on the disk.

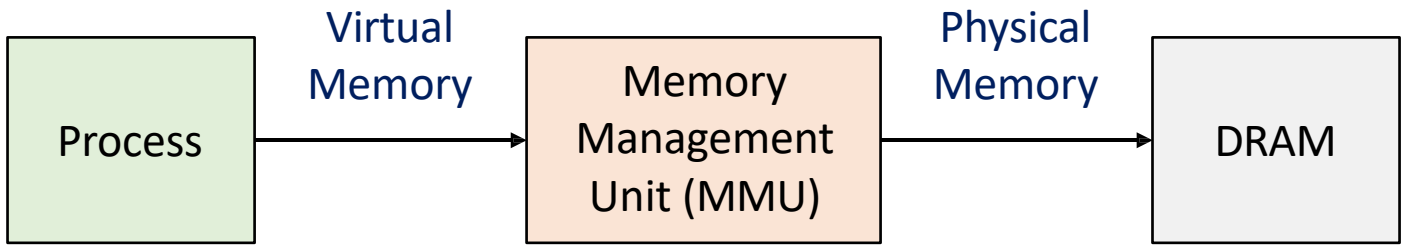
Virtual Address Space

- Enables **sparse** address spaces with holes left for growth, dynamically linked libraries, etc
- **System libraries** shared via mapping into virtual address space
- **Shared memory** by mapping pages read-write into virtual address space
  - Pages can be shared during `fork()`, speeding process creation

Memory Management Unit

At any instant in time computers are running multiple processes, each with its own address space. (Processes are described in the next section.) It would be too expensive to dedicate a full address space worth of memory for each process, especially because many processes use only a small part of their address space. Hence, there must be a means of sharing a smaller amount of physical memory among many processes.

One way to do this, *virtual memory*, divides physical memory into blocks and allocates them to different processes. Inherent in such an approach must be a *protection* scheme that restricts a process to the blocks belonging only to that process. Most forms of virtual memory also reduce the time to start a program, because not all code and data need be in physical memory before a program can begin.



Program Relocation

Virtual memory was invented to relieve programmers of this burden; it automatically manages the two levels of the memory hierarchy represented by main memory and secondary storage. Figure B.19 shows the mapping of virtual memory to physical memory for a program with four pages.

In addition to sharing protected memory space and automatically managing the memory hierarchy, virtual memory also simplifies loading the program for execution. Called *relocation*, this mechanism allows the same program to run in any location in physical memory. The program in Figure B.19 can be placed anywhere in physical memory or disk just by changing the mapping between them. (Prior to the popularity of virtual memory, processors would include a relocation register just for that purpose.) An alternative to a hardware solution would be software that changed all addresses in a program each time it was run.

Aside: Real-world ISAs – RISC-V

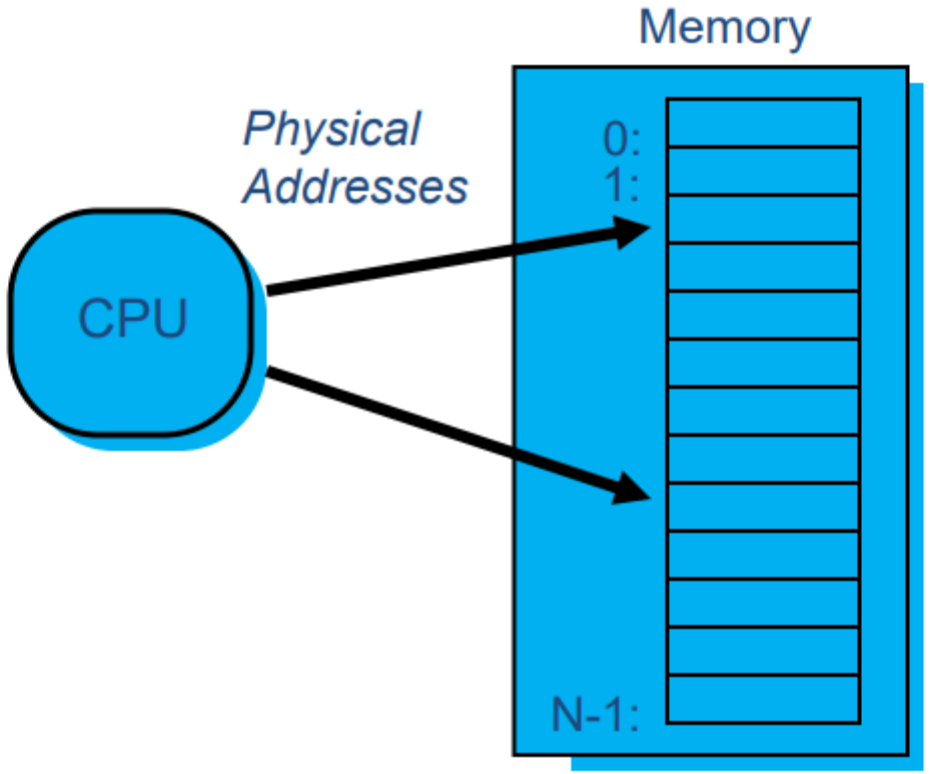
- RISC-V defines three different virtual memory specifications
  - Sv32: Two level hierarchy, 32-bit virtual, 34-bit physical address
  - Sv39: Three level, 39-bit virtual, 56-bit physical address
  - Sv48: Four level, 48-bit virtual, 56-bit physical address
  - Up to chip designers to choose what to support
  - OS can learn which is implemented by reading the CSR "satp" (Supervisor Address Translation and Protection)
  - Page table base register is a CSR, "sptbr" (Supervisor Page Table Base Register)
- Some specs have different PTE sizes
  - Sv32 has 32 bit PTEs (34-bit physical address space – 11 bit offset < 32 bits)
  - Sv39 and Sv48 has 64-bit PTEs

Mechanism of Virtual Memory

Computer with Physical Memory

Examples:

- most Cray machines, early PCs, many embedded systems, etc.



Addresses generated by the CPU correspond directly to bytes in physical memory

Virtualizing Main Memory

How do multiple apps (and the OS) share main memory?
Goal: each application thinks it has private memory

App's insn/data footprint > main memory ?

- **Requires main memory to act like a cache**
  - With disk as next level in memory hierarchy (slow)
  - Write-back, write-allocate, large blocks or "pages"

Solution:

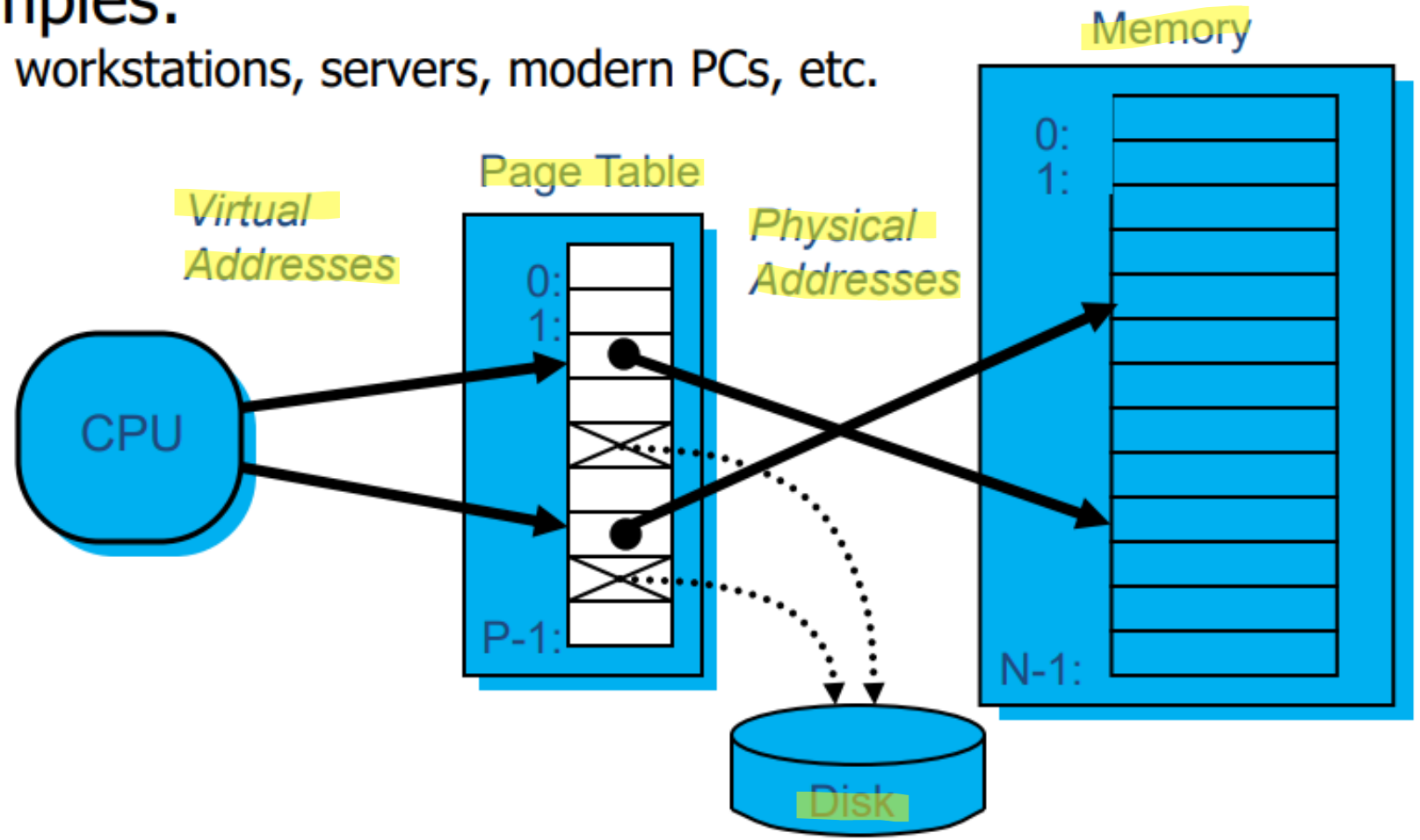
- Part #1: treat memory as a "cache"
- Part #2: add a level of indirection (address translation)

Parameter	I$/D$	L2	Main Memory
t_{hit}	2ns	10ns	**30ns**
t_{miss}	**10ns**	**30ns**	**10ms (10M ns)**
Capacity	8–64KB	128KB–2MB	**64MB–64GB**
Block size	16–32B	32–256B	**4+KB**
Assoc./Repl.	1–4, NMRU	4–16, NMRU	**Full, "working set"**

Computer With Virtual Memory

Examples:

- workstations, servers, modern PCs, etc.



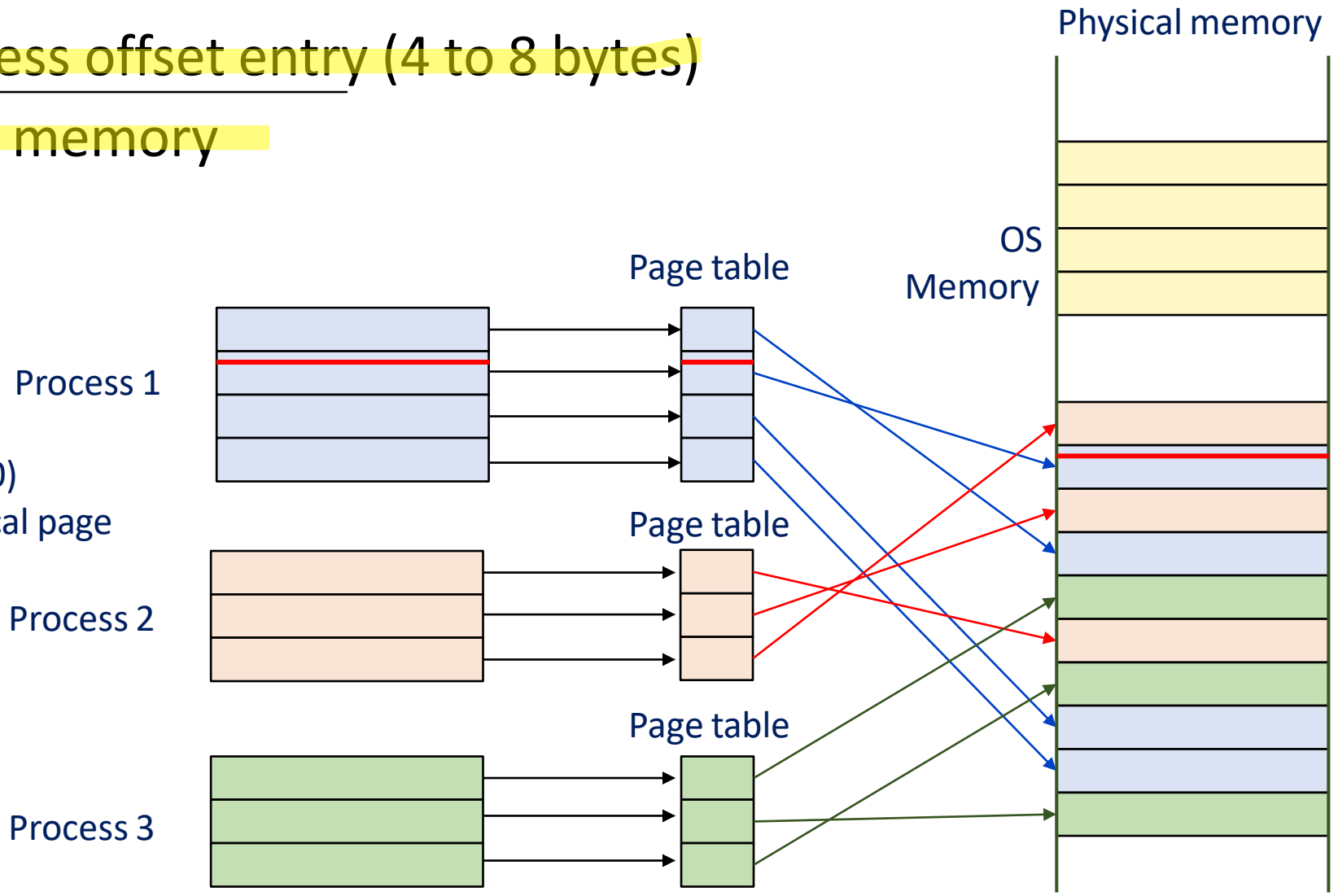
Address Translation: Hardware converts virtual addresses to physical addresses via OS-managed lookup table (page table)

Private Virtual Address Space per Process

- Each process has a page table
- Page table has an address offset entry (4 to 8 bytes) for each page of process memory

Example:
Process 1 issues write to **0x1100**
0x1100 is offset **0x100 in page 2** (0x1000)
Page table entry 2 is consulted for physical page

"Page table walk" handled in hardware by MMU

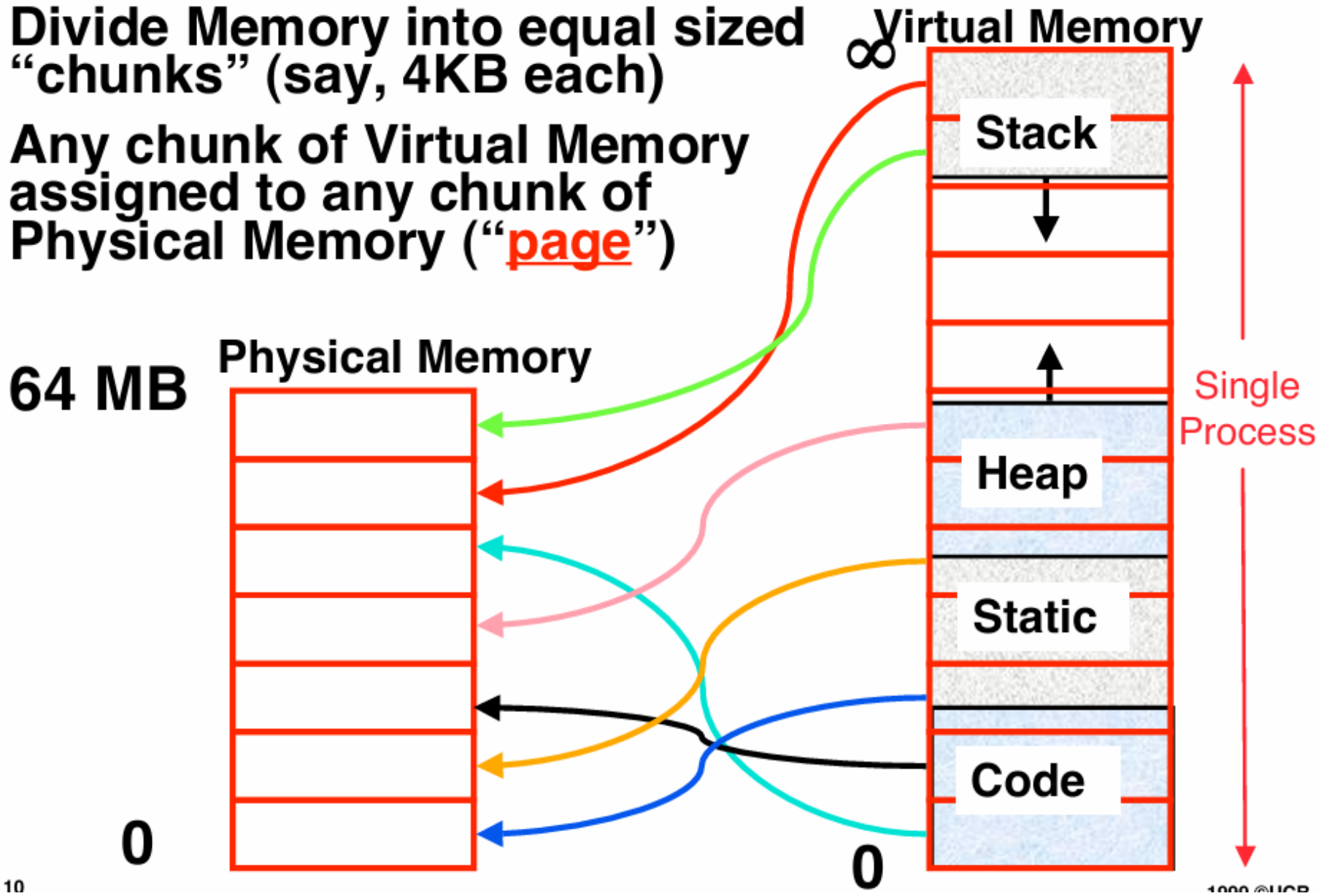


Virtual Memory Terminology

- Each process has its own private "virtual address space" (e.g., 2^{32} Bytes); CPU actually generates "virtual addresses"
- Each computer has a "physical address space" (e.g., 128 MegaBytes DRAM); also called "real memory"
- Address translation: mapping virtual addresses to physical addresses
  - Allows multiple programs to use (different chunks of physical) memory at same time
  - Also allows some chunks of virtual memory to be represented on disk, not in main memory (to exploit memory hierarchy)

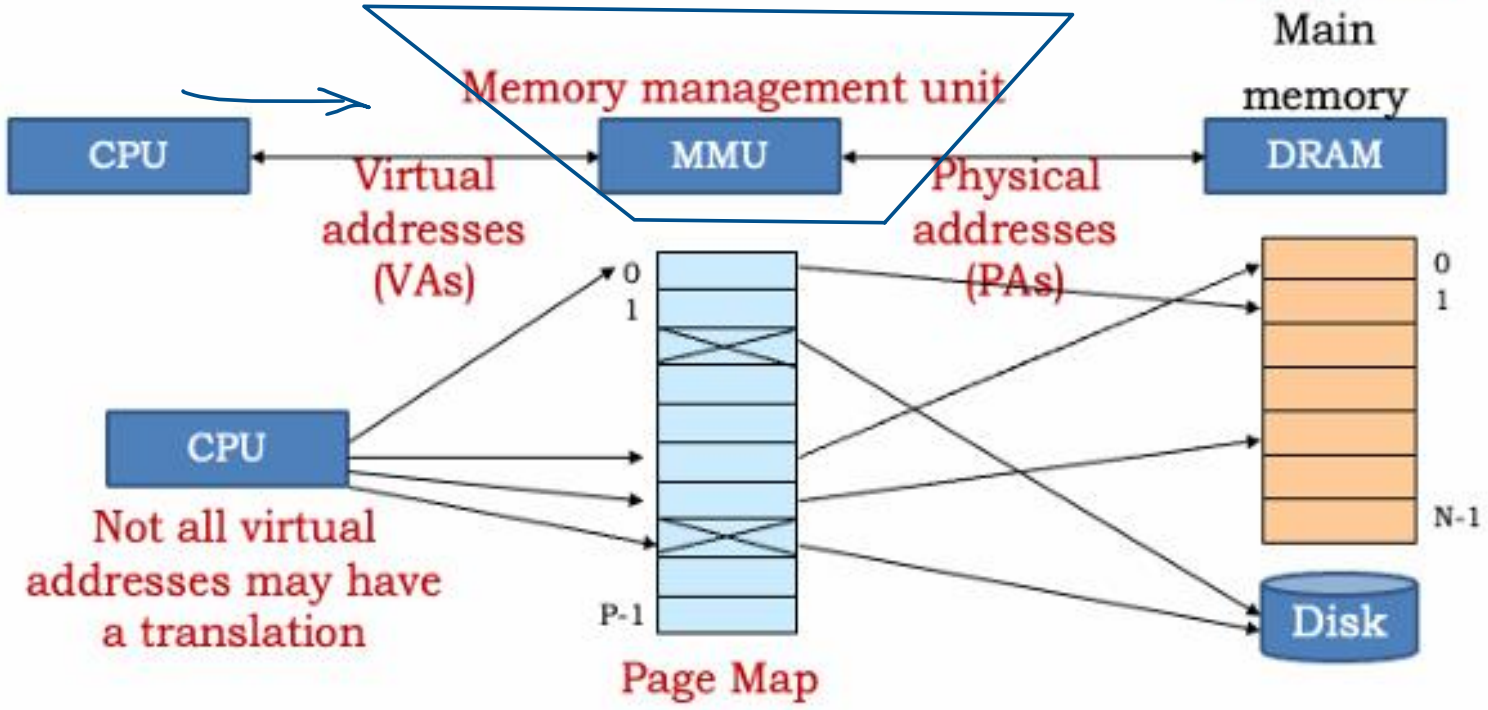
Mapping VM to PM

- Divide Memory into equal sized "chunks" (say, 4KB each)
- Any chunk of Virtual Memory assigned to any chunk of Physical Memory ("page")

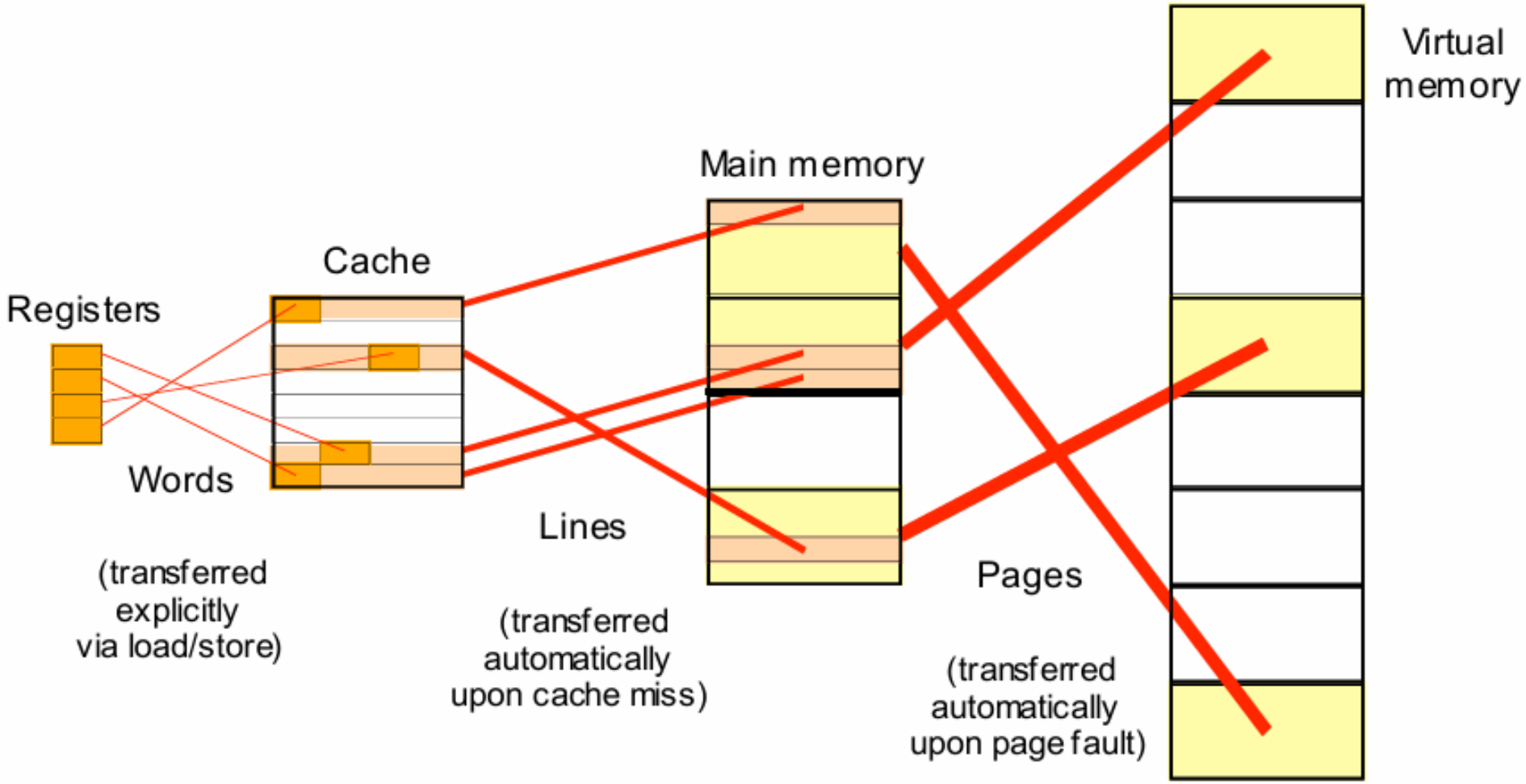


CPU, MMU and VM - Organization

- Two kinds of addresses:
  - CPU uses virtual addresses
  - Main memory uses physical addresses
- Hardware translates virtual addresses to physical addresses via an operating system (OS)-managed table, the page map



Data Movement in Memory Hierarchy including VM



Data movement in a memory hierarchy.

Comparing Cache and VM

Cache	*Virtual Memory*
Block or Line	Page
Miss	Page Fault
Block Size: 32-64B	Page Size: 4K-16KB
Placement: Direct Mapped, N-way Set Associative	Fully Associative
Replacement: LRU or Random	Least Recently Used (LRU) approximation
Write Thru or Back	Write Back
How Managed: Hardware	Hardware + Software (Operating System)

Virtual Memory Implementation

VA to PA Mapping using Page Table

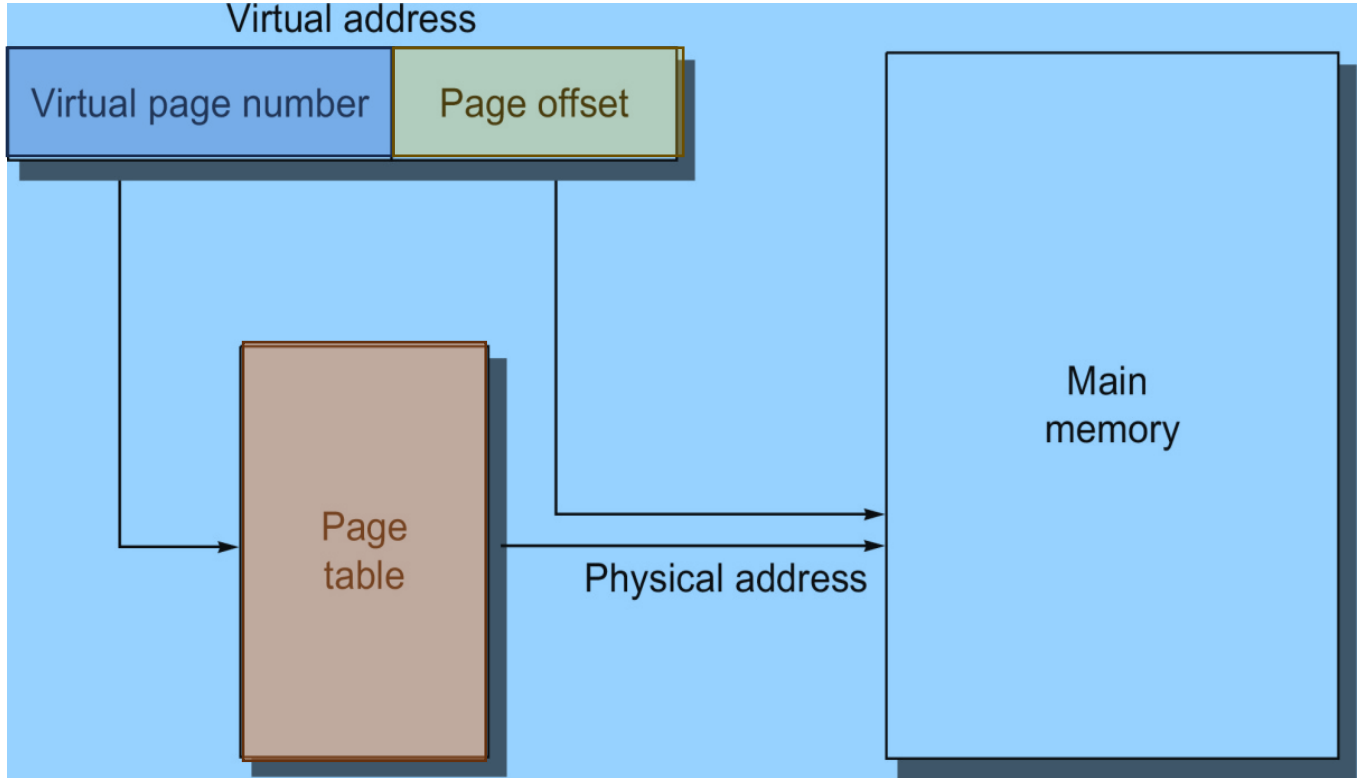
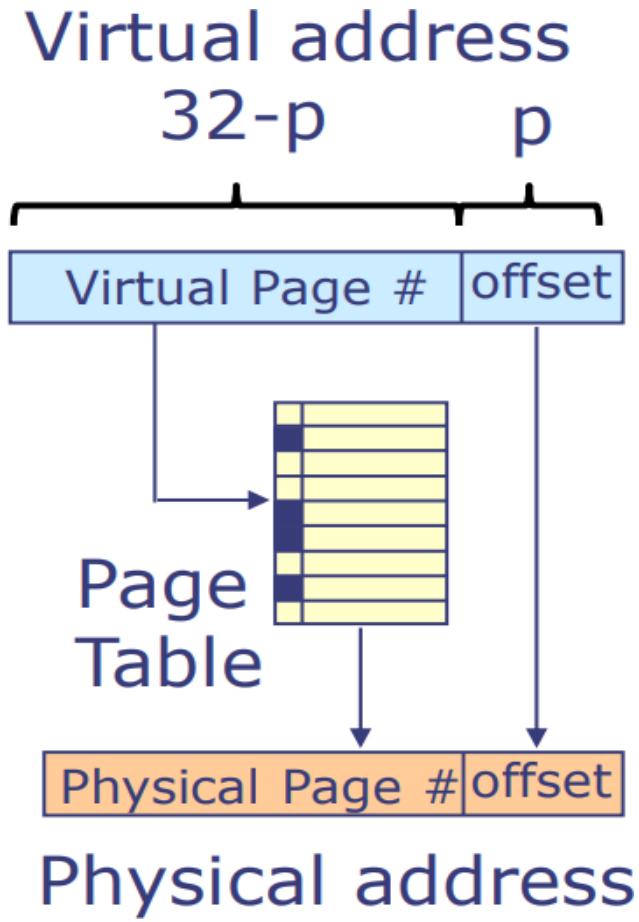


Figure B.23 The mapping of a virtual address to a physical address via a page table

Paged Virtual Memory

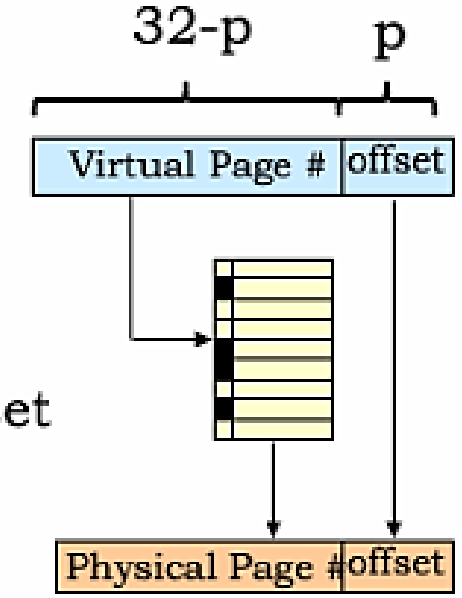
- Physical memory is divided into fixed-size units called pages
  - Most modern architectures use 4 KiB pages
- Virtual address is interpreted as a pair of <virtual page offset, and page internal offset>
- Use a per-process page table to translate virtual page offset to physical page offset
  - Page table contains the physical page number (i.e., starting physical address) for each virtual page



Who does this translation? Processor hardware, or OS software?

VM Implementation through Paging

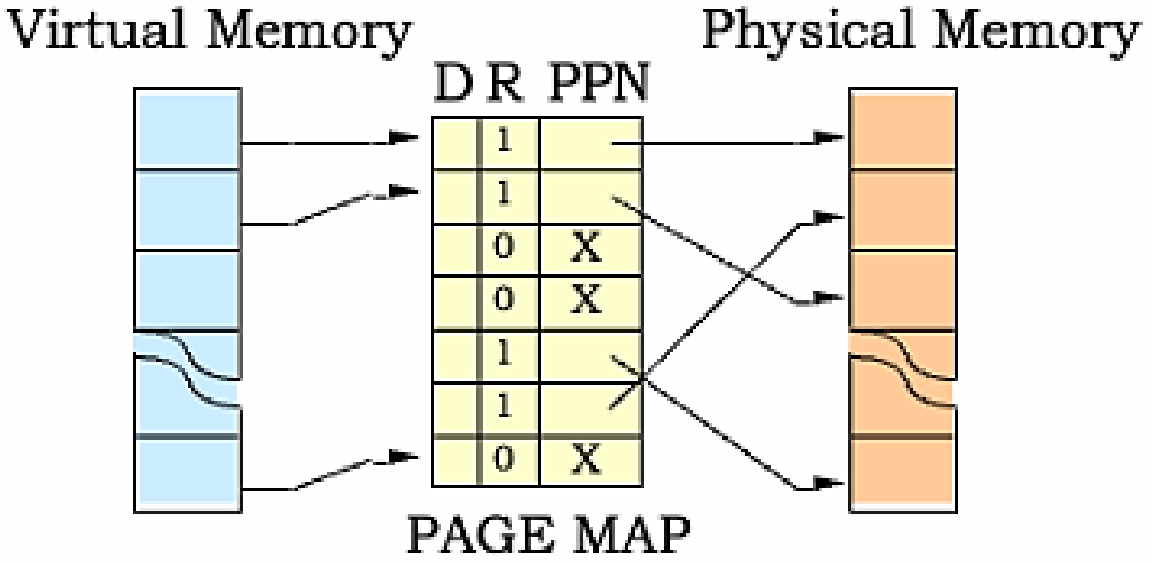
- Divide physical memory in fixed-size blocks, called pages
  - Typical page size (2^p): 4KB -16 KB
  - Virtual address: Virtual page number + offset bits
  - Physical address: Physical page number + offset bits
  - Why use lower bits as offset?



- MMU maps virtual pages to physical pages
  - Use page map to perform translation
  - Cause a page fault (a miss) if virtual page is not resident in physical memory.

Using main memory as a page cache = *paging* or *demand paging*

Simple Page Map Design

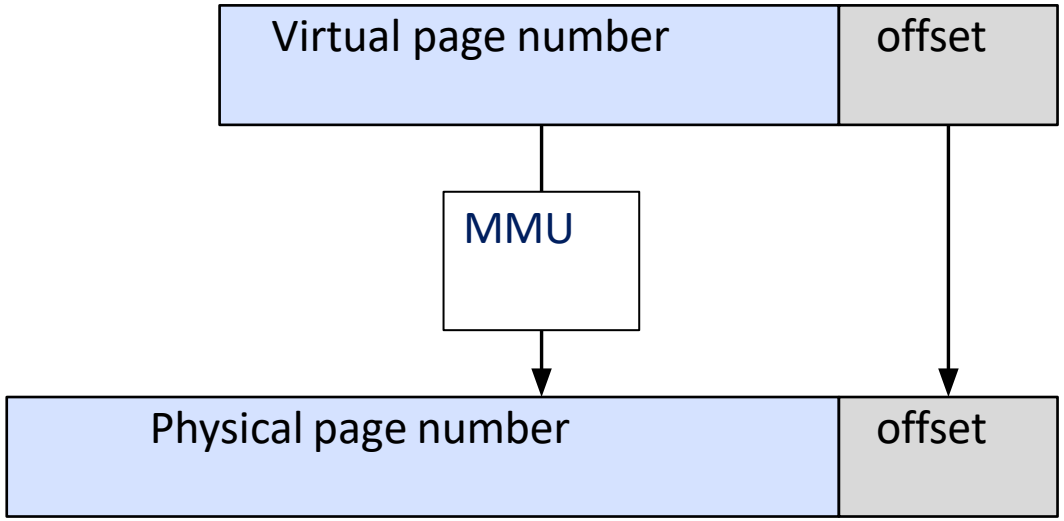


One entry per virtual page

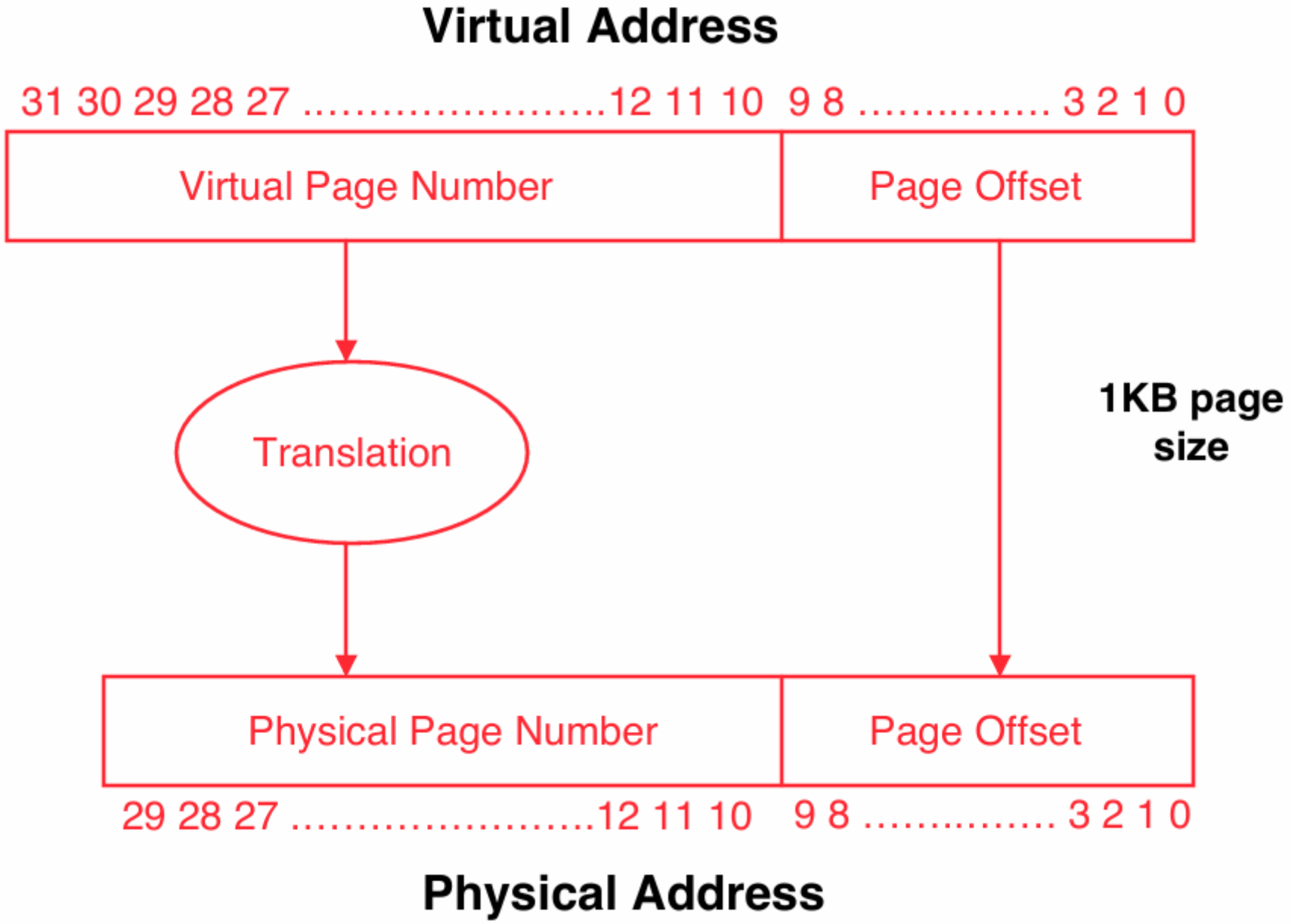
- Resident bit R = 1 for pages stored in RAM, or 0 for non-resident (disk or unallocated). Page fault when R = 0
- Contains physical page number (PPN) of each resident page
- Dirty bit D = 1 if we've changed this page since loading it from disk (and therefore need to write it to disk when it's replaced)

Virtual and Physical Address Sizes

- The size of virtual and physical address spaces can be different!
  - Implementations may have larger physical address space than virtual
  - Since multiple processes share a physical memory
- Typically design decision regarding resource efficiency



Mapping Virtual to Physical Address



Perform Address Translation

How to Perform Address Translation?

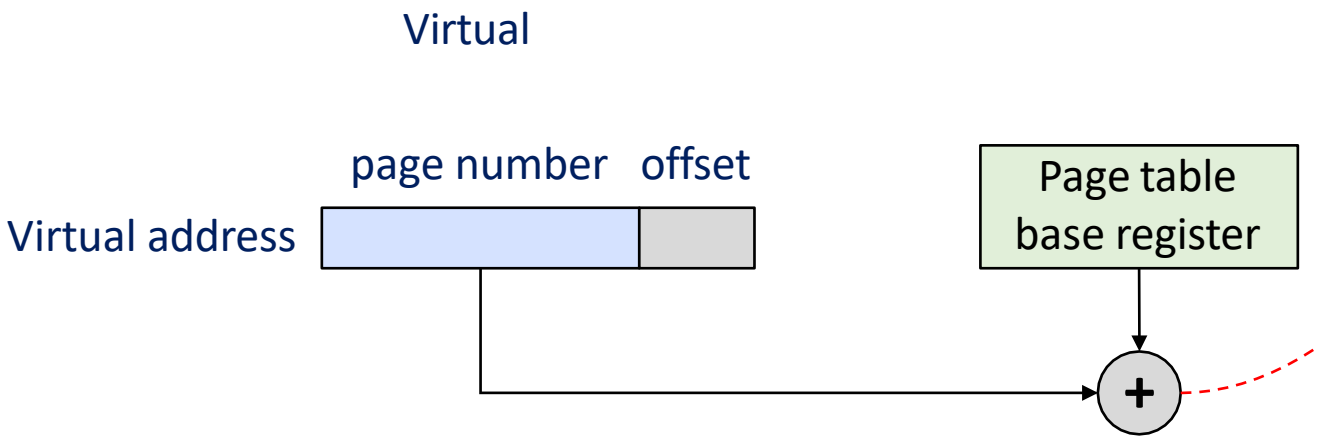
- VM divides memory into equal sized pages
- Address translation relocates entire pages
  - offsets within the pages do not change
  - if make page size a power of two, the virtual address separates into two fields:

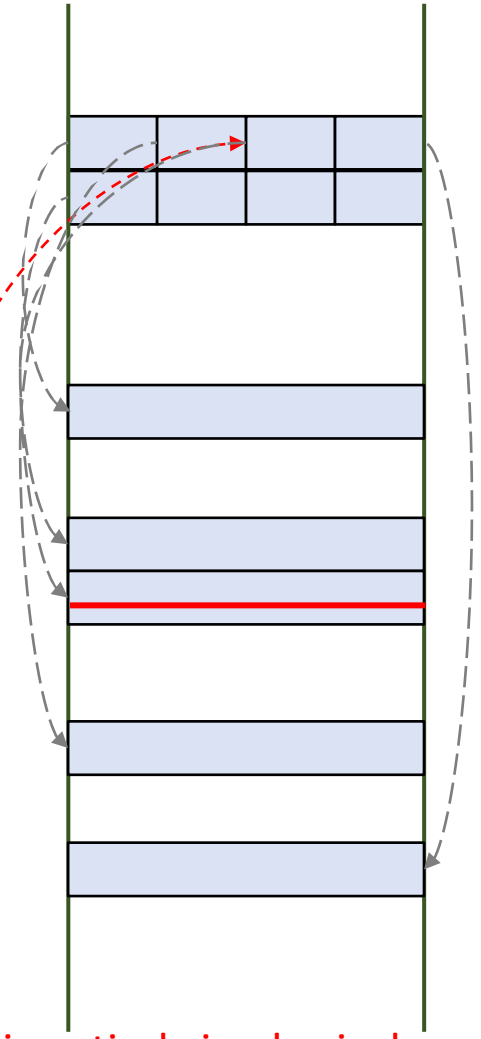


  - like cache index, offset fields

Page Table Implementation

- Physical page number
  = Mem[PT base + virtual page number]
- Physical address
  = Physical page number + offset
- Each cache miss (and DRAM access) involves **two physical memory access!**

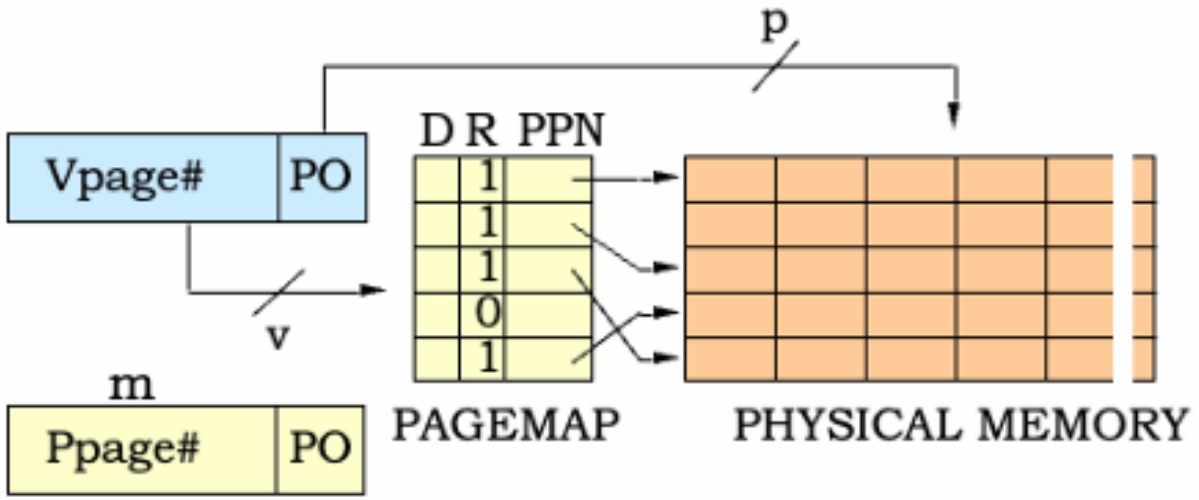




Page table is entirely in physical memory space!

Simple Page Map Arithmetic Example

Page Map Arithmetic

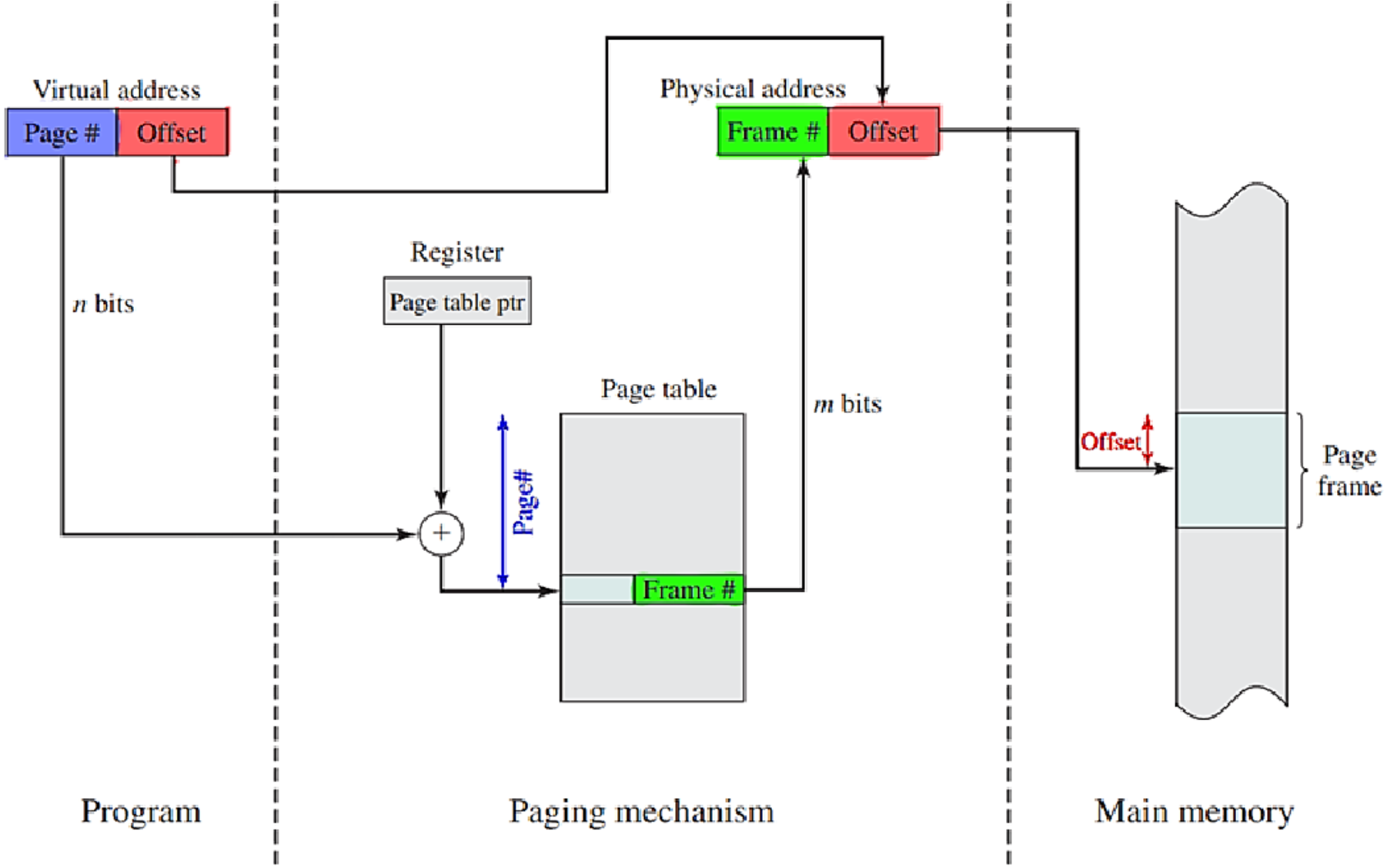


(v + p)	bits in virtual address
(m + p)	bits in physical address
2^v	number of VIRTUAL pages
2^m	number of PHYSICAL pages
2^p	bytes per physical page
2^{v+p}	bytes in virtual memory
2^{m+p}	bytes in physical memory
$(m+2)2^v$	bits in the page map

Typical page size: 4KB -16 KB
Typical (v+p): 32-64 bits
(4GB-16EB)
Typical (m+p): 30-40+ bits
(1GB-1TB)

Long virtual addresses allow ISAs to support larger memories → ISA longevity

Address Translation Mechanism

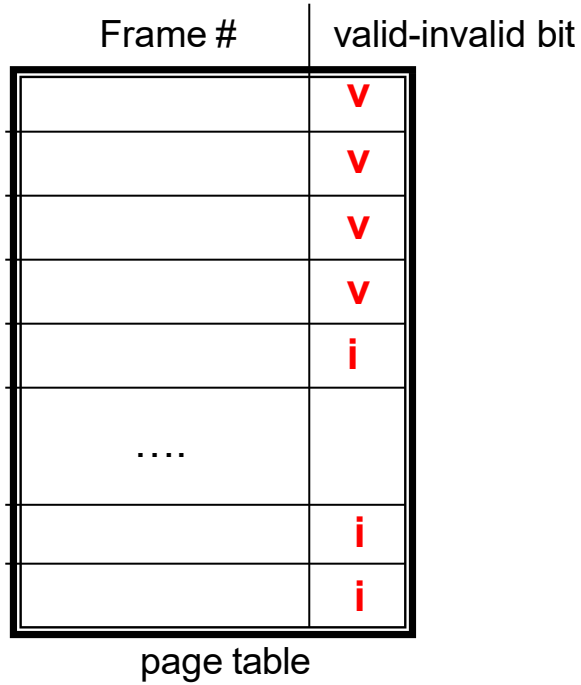


Address Translation Example



Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (v ⇒ in-memory – memory resident, i ⇒ not-in-memory)
- Initially valid–invalid bit is set to i on all entries
- Example of a page table snapshot:



- During address translation, if valid–invalid bit in page table entry is **i** ⇒ page fault

Pages and Segments

Virtual memory also encompasses several related techniques. Virtual memory systems can be categorized into two classes: those with fixed-size blocks, called *pages*, and those with variable-size blocks, called *segments*. Pages are typically fixed at 4096–8192 bytes, while segment size varies. The largest segment supported on any processor ranges from 2^{16} bytes up to 2^{32} bytes; the smallest segment is 1 byte. Figure B.21 shows how the two approaches might divide code and data.

The decision to use paged virtual memory versus segmented virtual memory affects the processor. Paged addressing has a single fixed-size address divided into page number and offset within a page, analogous to cache addressing. A single address does not work for segmented addresses; the variable size of segments requires 1 word for a segment number and 1 word for an offset within a segment, for a total of 2 words. An unsegmented address space is simpler for the compiler.

Page vs Segment

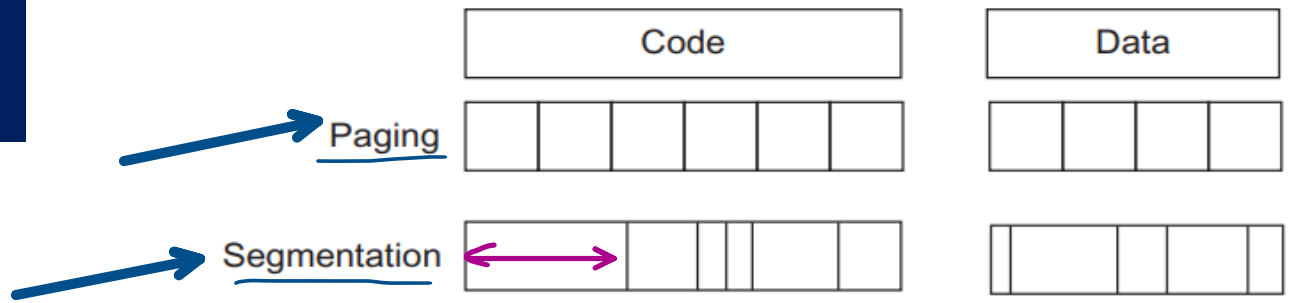


Figure B.21 Example of how paging and segmentation divide a program.

	Page	Segment
Words per address	One	Two (segment and offset)
Programmer visible?	Invisible to application programmer	May be visible to application programmer
Replacing a block	Trivial (all blocks are the same size)	Difficult (must find contiguous, variable-size, unused portion of main memory)
Memory use inefficiency	Internal fragmentation (unused portion of page)	External fragmentation (unused pieces of main memory)
Efficient disk traffic	Yes (adjust page size to balance access time and transfer time)	Not always (small segments may transfer just a few bytes)

Figure B.22 Paging versus segmentation. Both can waste memory, depending on the block size and how well the segments fit together in main memory. Programming languages with unrestricted pointers require both the segment and the address to be passed. A hybrid approach, called *paged segments*, shoots for the best of both worlds: segments are composed of pages, so replacing a block is easy, yet a segment may be treated as a logical unit.

Paging vs. Segmentation

- Page table makes it possible to store pages non-contiguously
  - Drastically reduces fragmentation issue
  - Processes can be mapped to share some physical pages, often mapped to different virtual addresses

- But Page table is MUCH larger than base+bound registers
  - For a 4 GiB address space, each process needs 4 GiB/4 KiB = 1 Million entries
  - Assuming 4 byte entries, 4 MiB per process
  - Base+bound registers were 2 * 4 bytes = 8 bytes per process!
  - No way 4 MiB per process can fit on-chip, meaning page tables must reside in off-chip DRAM

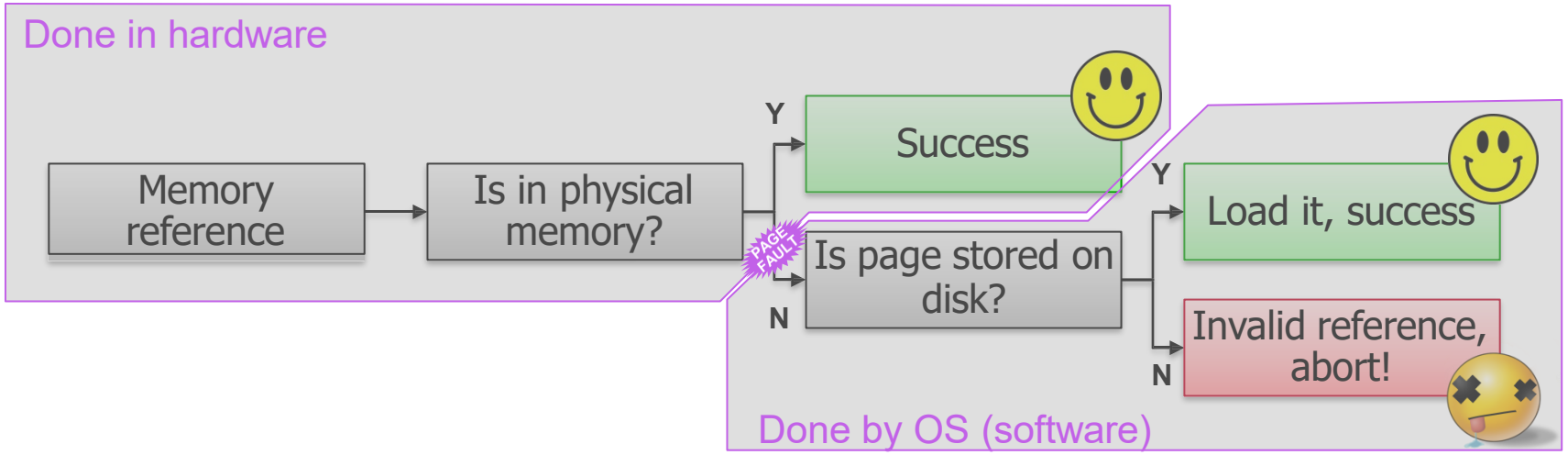
Introduce Demand Paging

Basic idea:

- Start with all virtual pages in secondary storage, MMU "empty", ie, there are no pages resident in physical memory
- Begin running program… each VA "mapped" to a PA
  - Reference to RAM-resident page: RAM accessed by hardware
  - Reference to a non-resident page: page fault, which traps to software handler, which
    - Fetches missing page from DISK into RAM
    - Adjusts MMU to map newly-loaded virtual page directly in RAM
    - If RAM is full, may have to replace ("swap out") some little-used page to free up RAM for the new page.
- Working set incrementally loaded via page faults, gradually evolves as pages are replaced…

Demand Paging

- On process load:
  - Rather than bring entire process into memory, bring it in page-by-page as needed
  - Less I/O needed, no unnecessary I/O

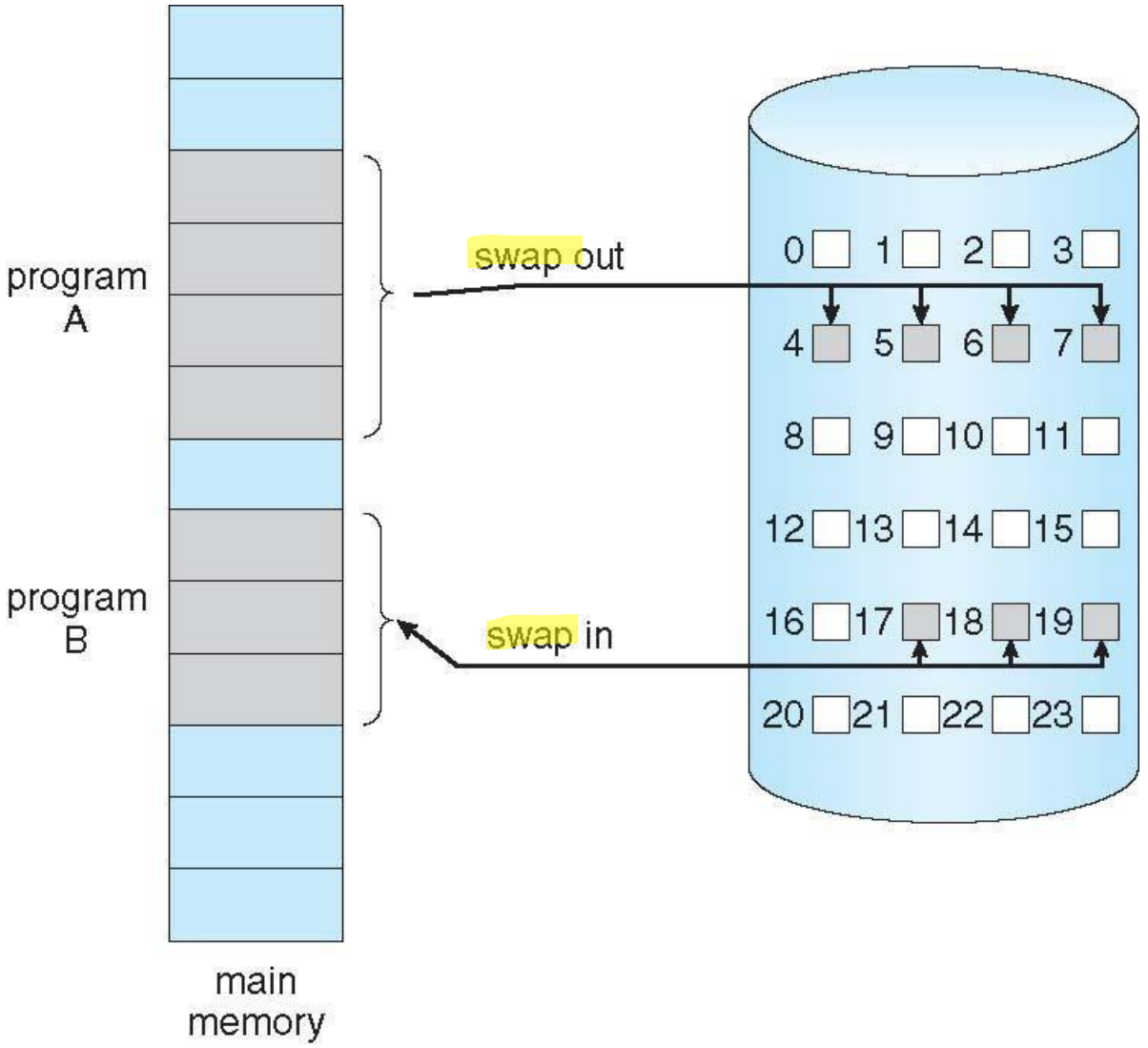


- The swapper process is called the **pager**

Aspects of Demand Paging

- Extreme case – start process with no pages in memory
  - OS sets instruction pointer to first instruction of process, non-memory-resident -> page fault
  - And for every other process pages on first access
  - **Pure demand paging**
- Actually, a given instruction could access multiple pages -> multiple page faults
  - Pain decreased because of **locality of reference**
- Hardware support needed for demand paging
  - Page table with valid / invalid bit
  - Secondary memory (swap device with **swap space**)
  - Instruction restart

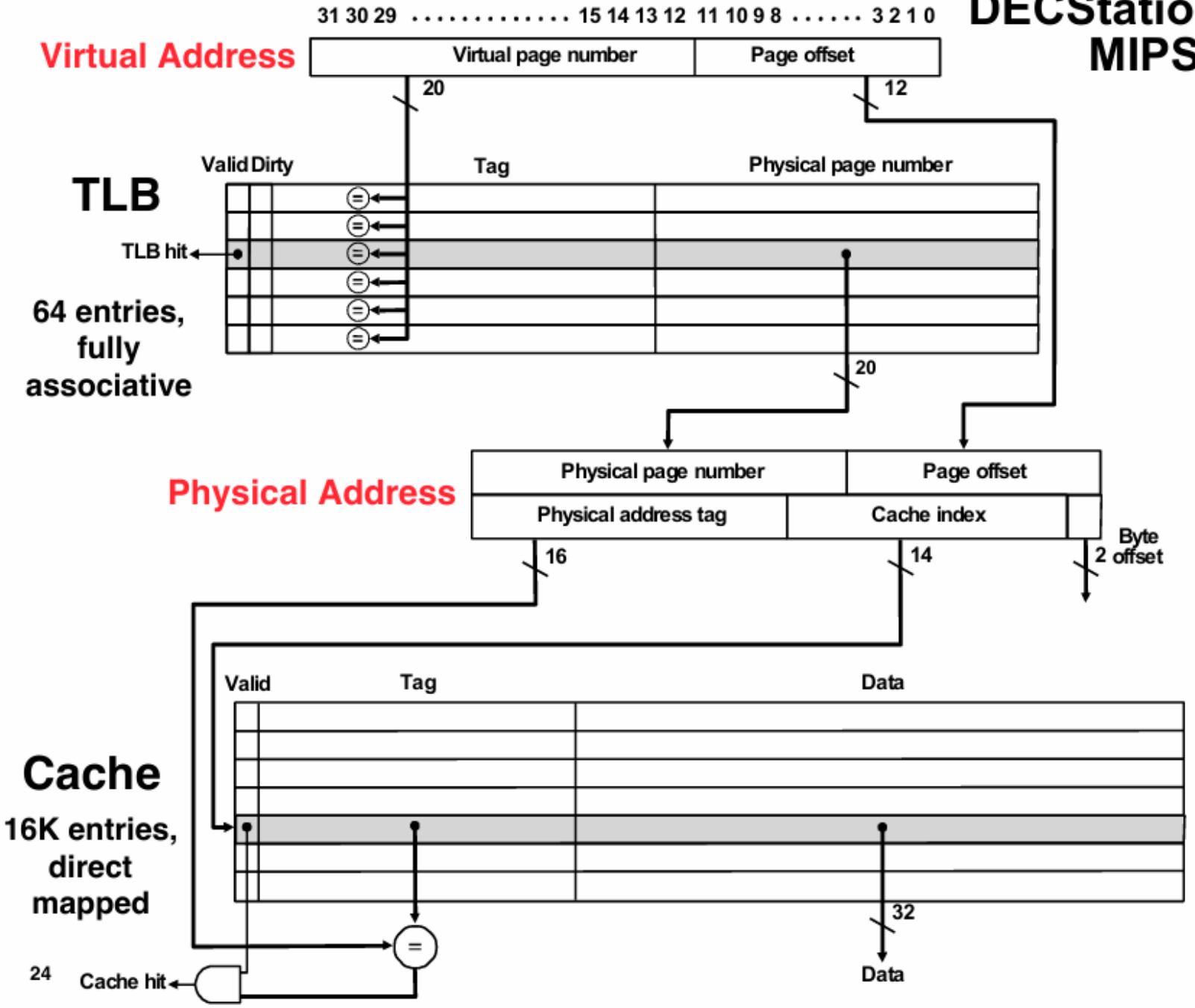
Transfer of a Paged Memory to Contiguous Disk Space



Aside: Real-world ISAs – RISC-V

- RISC-V defines three different virtual memory specifications
  - Sv32: Two level hierarchy, 32-bit virtual, 34-bit physical address
  - Sv39: Three level, 39-bit virtual, 56-bit physical address
  - Sv48: Four level, 48-bit virtual, 56-bit physical address
  - Up to chip designers to choose what to support
  - OS can learn which is implemented by reading the CSR "satp" (Supervisor Address Translation and Protection)
  - Page table base register is a CSR, "sptbr" (Supervisor Page Table Base Register)
- Some specs have different PTE sizes
  - Sv32 has 32 bit PTEs (34-bit physical address space – 11 bit offset < 32 bits)
  - Sv39 and Sv48 has 64-bit PTEs

DECStation 3100/ MIPS R2000



1999 ©UCB

Translation Lookaside Buffer TLB

Fast Address Translation

Techniques for Fast Address Translation

Page tables are usually so large that they are stored in main memory and are sometimes paged themselves. Paging means that every memory access logically takes at least twice as long, with one memory access to obtain the physical address and a second access to get the data. As mentioned in Chapter 2, we use locality to avoid the extra memory access. By keeping address translations in a special cache, a memory access rarely requires a second access to translate the data. This special address translation cache is referred to as a *translation look aside buffer* (TLB), also called a *translation buffer* (TB).

A TLB entry is like a cache entry where the tag holds portions of the virtual address and the data portion holds a physical page frame number, protection field, valid bit, and usually a use bit and dirty bit. To change the physical page frame number or protection of an entry in the page table, the operating system must make sure the old entry is not in the TLB; otherwise, the system won't behave properly. Note that this dirty bit means the corresponding *page* is dirty, not that the address translation in the TLB is dirty nor that a particular block in the data cache is dirty. The operating system resets these bits by changing the value in the page table and then invalidates the corresponding TLB entry. When the entry is reloaded from the page table, the TLB gets an accurate copy of the bits.

Translation Lookaside Buffer (TLB)

- Solution: Use some on-chip memory to cache virtual-to-physical address translations
  - TLB hit -> Immediate translation
  - TLB miss -> Must walk page table and populate TLB for later
- Similarly to data caches, TLB is:
  - indexed by a subset of the page number
  - tagged by the remainder of the page number
  - Involves trade-offs involving associativity
  - (Mostly) transparent to software
  - Often multi-level structure

e.g., x86 Haswell has L1 TLB of 64 items, 4-way, and L2 TLB of 1024 items, 8-way

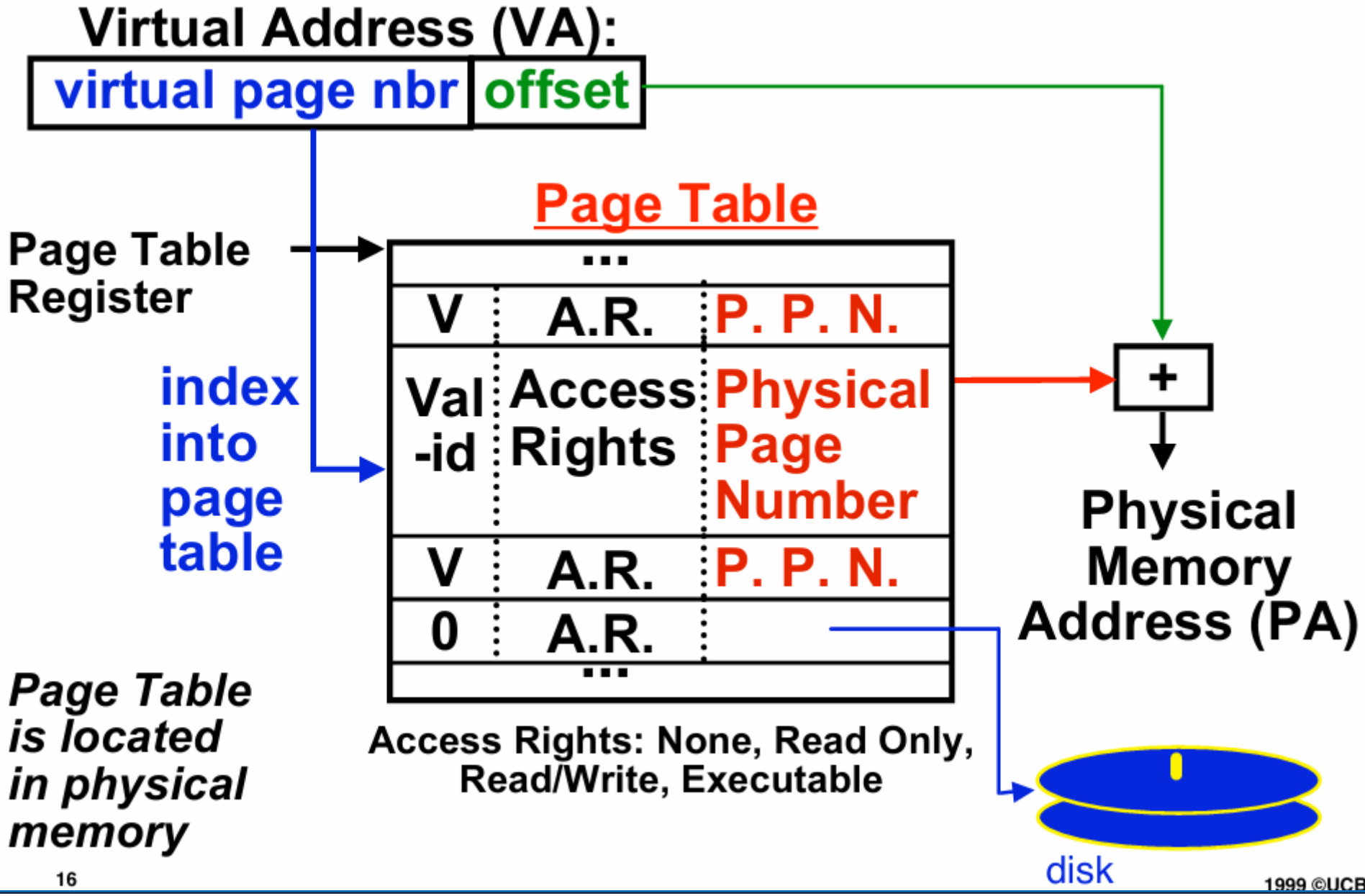
Hierarchical page tables

- Operating system must be aware of the hierarchy
  - Construct a level-1 page table per process
  - Unallocated virtual memory access causes an exception, and the exception handler creates level-2 page tables as required
  - Virtual memory access is a concerted effort btw OS/architecture!
- Multi-level page tables mean TLB misses result in more than two memory accesses
  - Importance of, and pressure on, TLB increases!

Address Translation

- Want fully associative page placement
- How to locate the physical page?
- Search impractical (too many pages)
- A page table is a data structure which contains the mapping of virtual pages to physical pages
  - There are several different ways, all up to the operating system, to keep this data around
- Each process running in the system has its own page table

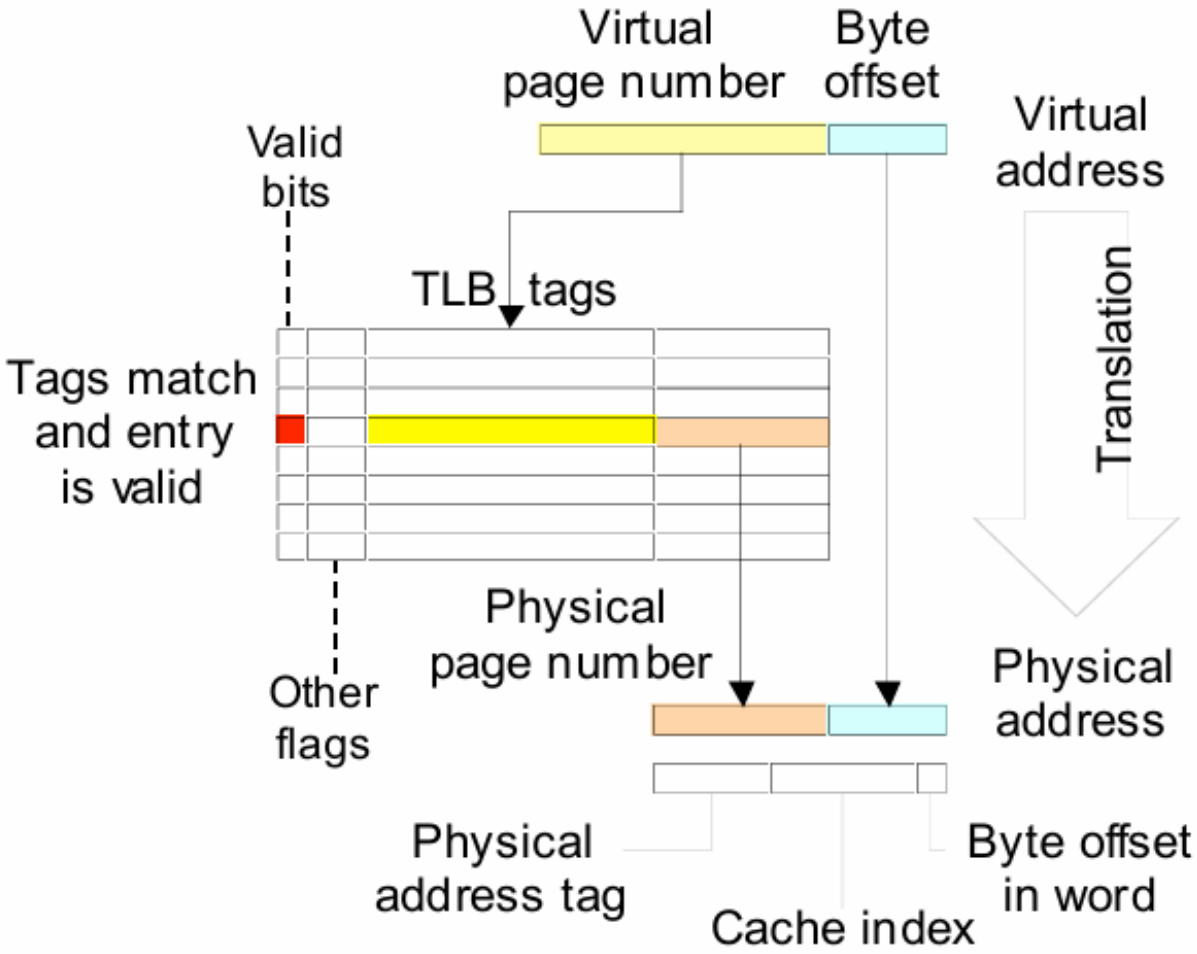
Address Translation – Page Table



How to Translate Fast?

- Problem: Virtual Memory requires two memory accesses!
  - one to translate Virtual Address into Physical Address (page table lookup)
  - one to transfer the actual data (cache hit)
  - But Page Table is in physical memory! => 2 main memory accesses!
- Observation: since there is locality in pages of data, must be locality in virtual addresses of those pages!
- Why not create a cache of virtual to physical address translations to make translation fast? (smaller is faster)
- For historical reasons, such a "page table cache" is called a Translation Lookaside Buffer, or TLB

Translation Look-aside Buffer



Virtual-to-physical address translation by a TLB and how the resulting physical address is used to access the cache memory.

Typical TLB Format

Virtual Page Nbr	Physical Page Nbr	Valid	Ref	Dirty	Access Rights

"tag" "data"

- TLB just a cache of the page table mappings

- Dirty: since use write back, need to know whether or not to write page to disk when replaced
- Ref: Used to calculate LRU on replacement

- TLB access time comparable to cache (much less than main memory access time)

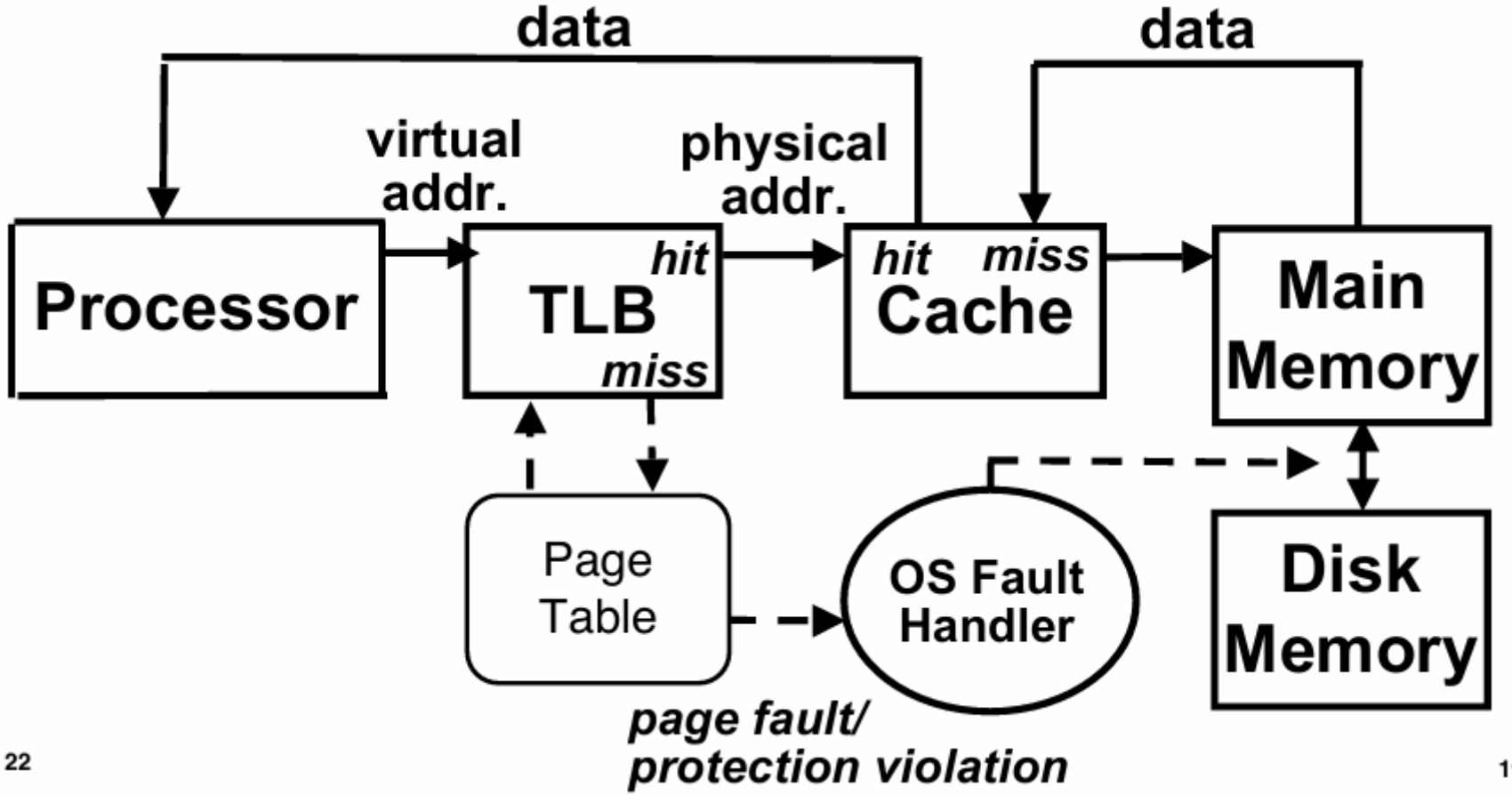
21

1999

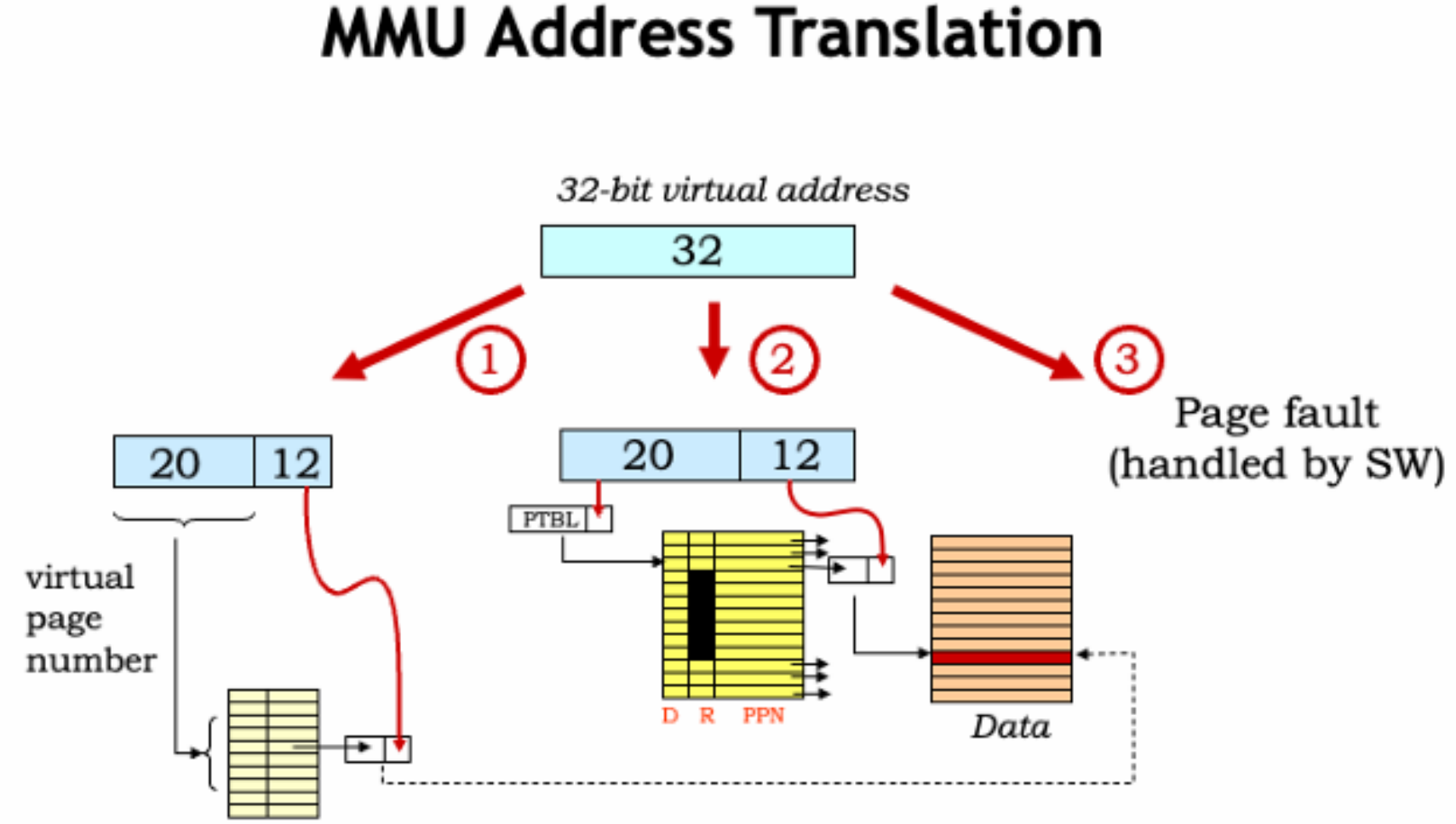
CPU, TLB, Cache and PM

- TLB is usually small, typically 32-4,096 entries

- Like any other cache, the TLB can be fully associative, set associative, or direct mapped



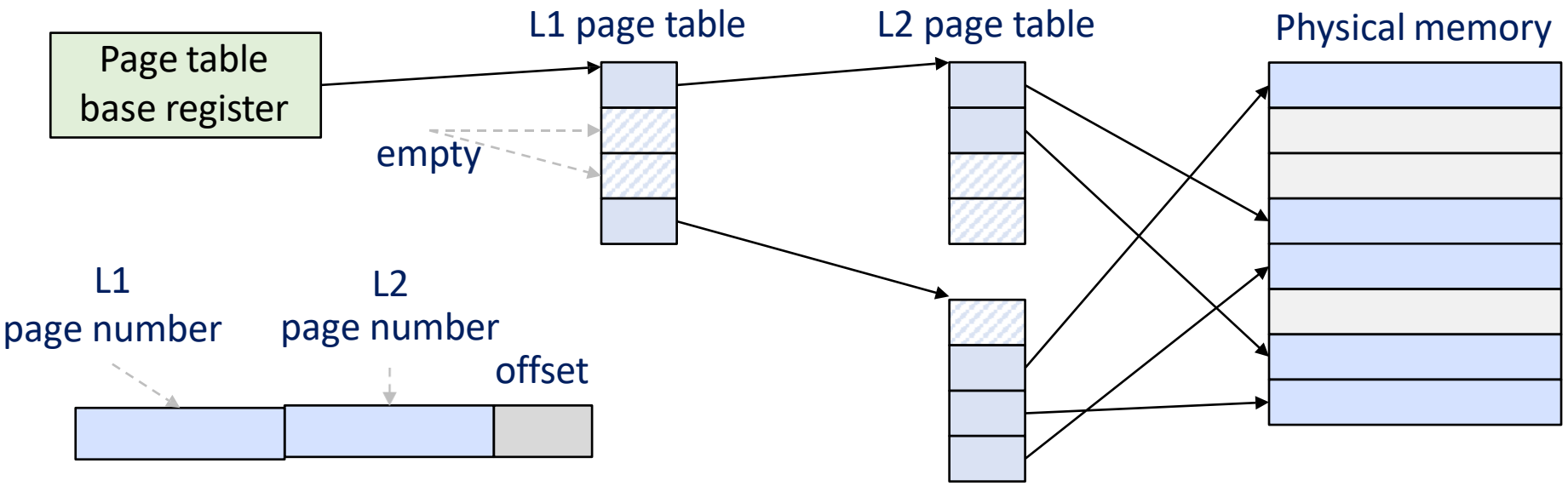
MMU Address Translation Operation



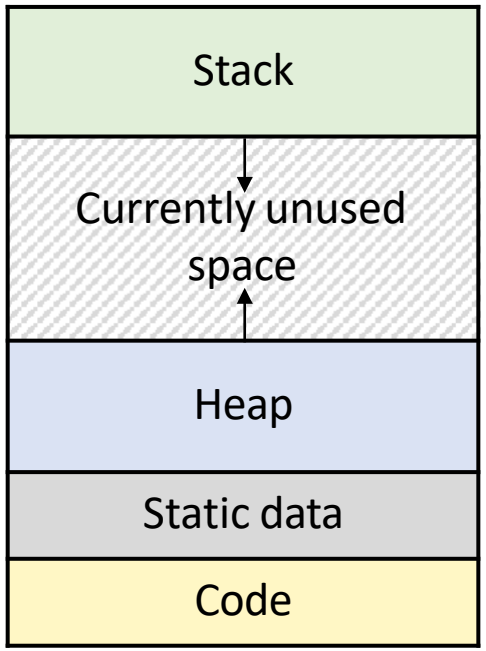
Look in TLB: VPN→PPN cache
Usually implemented as a small
fully-associative cache

Hierarchical Page Tables L1 and L2

- Most programs typically do not use all available virtual memory space
  - Meaning most of the page table is never used
- Hierarchical page tables exploit this to save space
  - Entries are only created when accessed
  - If L1 table entry is never accessed, no L2 page table created!



Process memory map



Opteron Data TLB Organization

Figure B.24 shows the Opteron data TLB organization, with each step of the translation labeled. This TLB uses fully associative placement; thus, the translation begins (steps 1 and 2) by sending the virtual address to all tags. Of course, the tag must be marked valid to allow a match. At the same time, the type of memory access is checked for a violation (also in step 2) against protection information in the TLB.

For reasons similar to those in the cache case, there is no need to include the 12 bits of the page offset in the TLB. The matching tag sends the corresponding physical address through effectively a 40:1 multiplexor (step 3). The page offset is then combined with the physical page frame to form a full physical address (step 4). The address size is 40 bits.

Address translation can easily be on the critical path determining the clock cycle of the processor, so the Opteron uses virtually addressed, physically tagged L1 caches.

Opteron Data TLB Figure

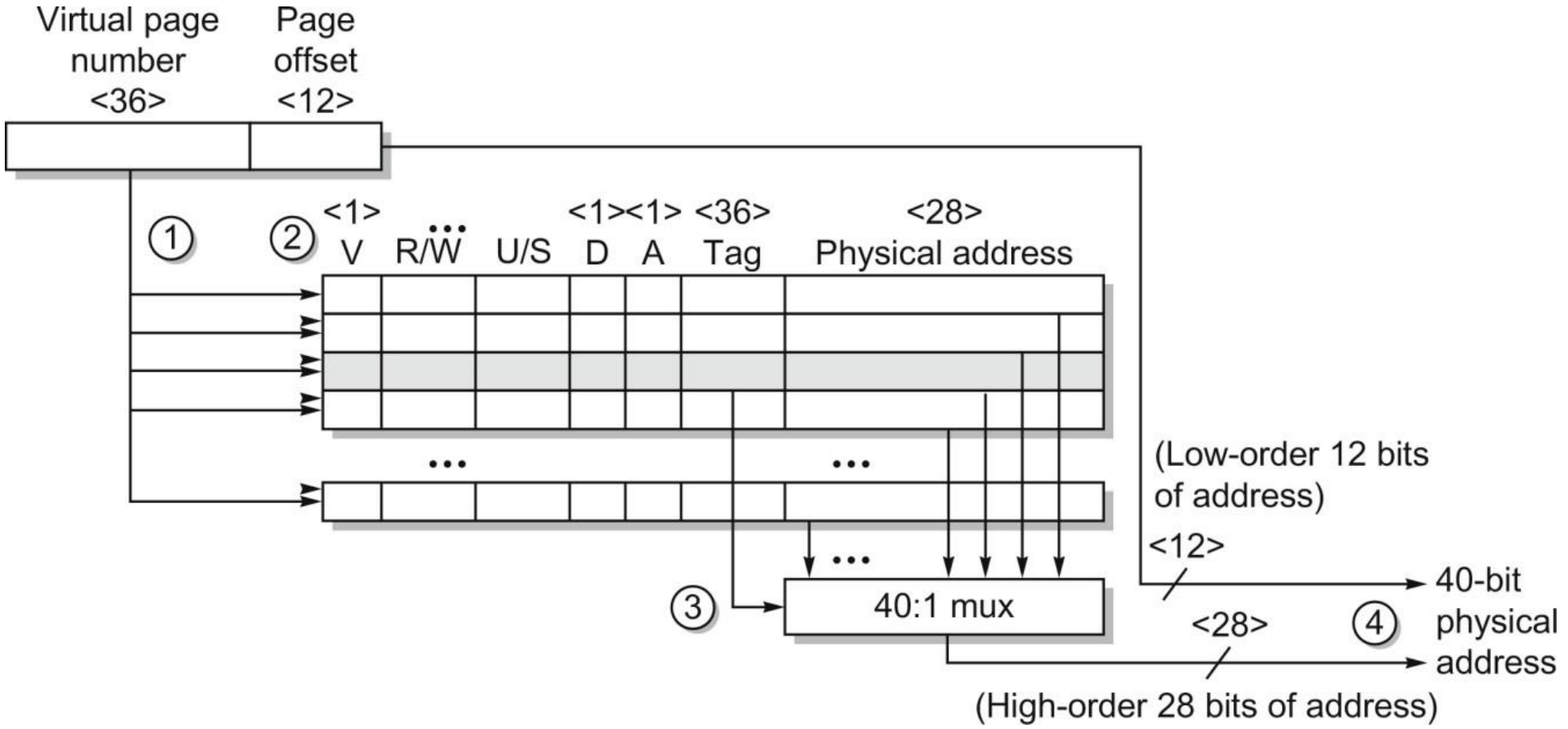


Figure B.24 Operation of the Opteron data TLB during address translation. The four steps of a TLB hit are shown as *circled numbers*. This TLB has 40 entries. Section B.5 describes the various protection and access fields of an Opteron page table entry.

Page Fault, Page Table Size

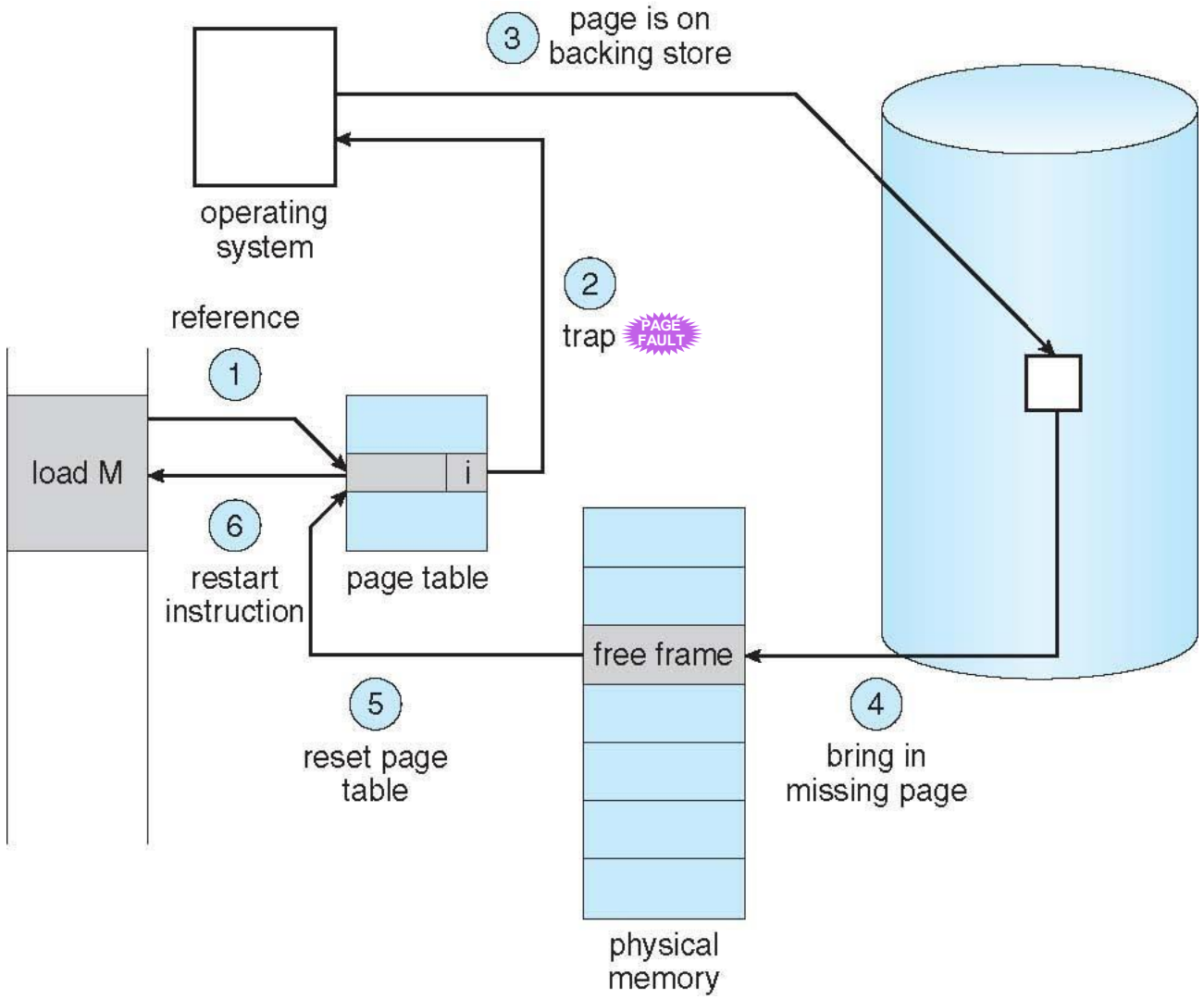
Handling Page Faults

- A page fault is like a cache miss
  - Must find page in lower level of hierarchy
- If valid bit is zero, the Physical Page Number points to a page on disk
- When OS starts new process, it creates space on disk for all the pages of the process, sets all valid bits in page table to zero, and all Physical Page Numbers to point to disk
  - called Demand Paging - pages of the process are loaded from disk only as needed

Page Fault

- Reference to a page with bit set to "i":
  Page fault
- OS looks at another table to decide:
  - Invalid reference? Abort.
- Just not in memory? Load it:
  - Get empty memory frame
  - Swap page into frame via scheduled disk operation
  - Reset tables to indicate page now in memory
    Set validation bit = "v"
  - Restart the instruction that caused the page fault

Steps in Handling a Page Fault - OS



Address Translation Mechanics

- The six questions
  - What? address translation
  - Why? compatibility, multi-programming, protection
  - How? page table
  - **Who performs it?**
  - **When?**
  - **Where does page table reside?**

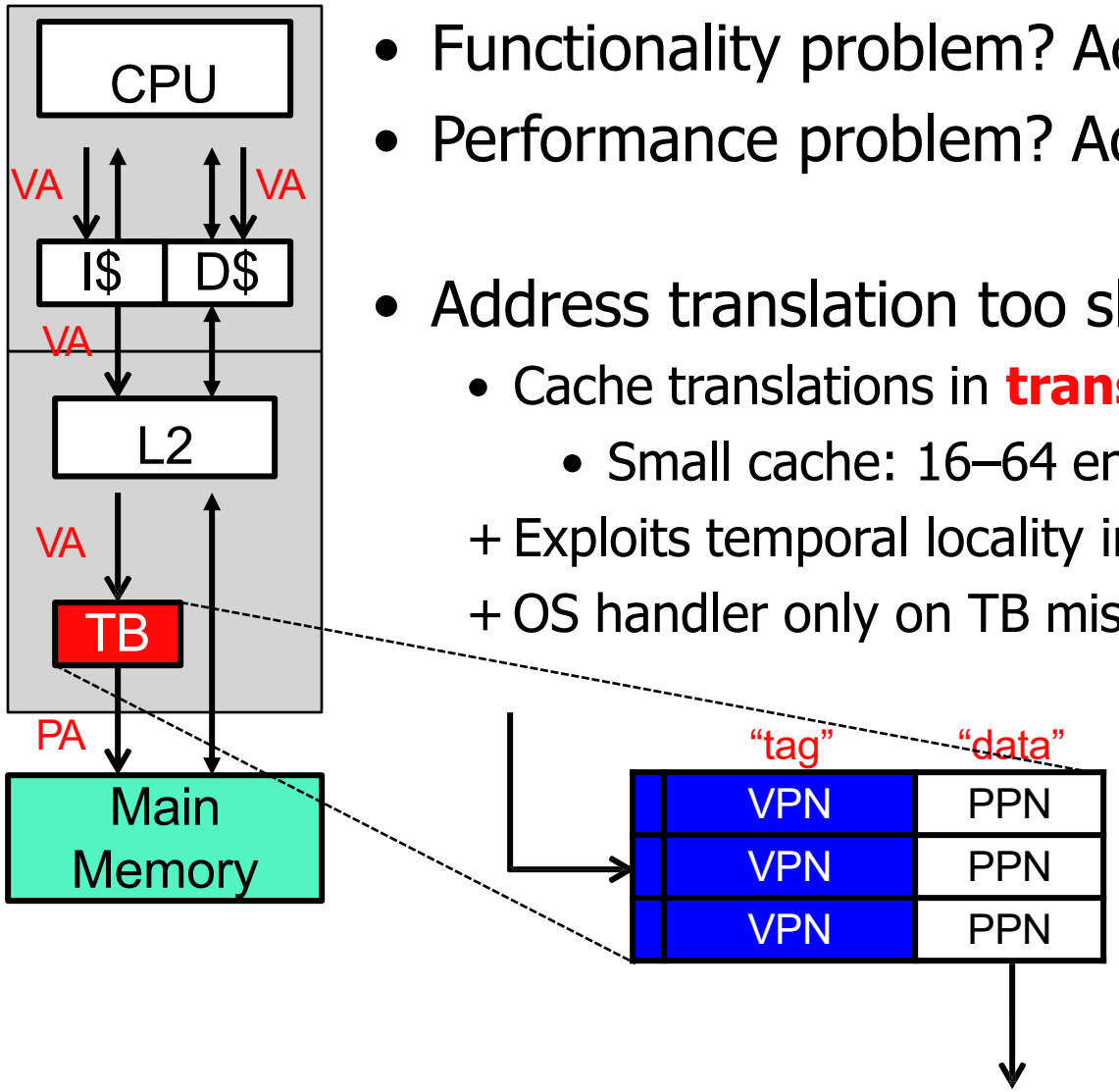
- Option I: process (program) translates its own addresses
  - Page table resides in process visible virtual address space
  - – Bad idea: implies that program (and programmer)…
    - …must know about physical addresses
      - Isn't that what virtual memory is designed to avoid?
    - …can forge physical addresses and mess with other programs
  - Translation on L2 miss or always? How would program know?

Who? Where? When? Take II

- Option II: **operating system (OS)** translates for process
  - Page table resides in OS virtual address space
  - \+ User-level processes cannot view/modify their own tables
  - \+ User-level processes need not know about physical addresses
  - Translation on L2 miss
    - – Otherwise, OS SYSCALL before any fetch, load, or store

- L2 miss: interrupt transfers control to OS handler
  - Handler translates VA by accessing process's page table
  - Accesses memory using PA
  - Returns to user process when L2 fill completes
  - – Still slow: added interrupt handler and PT lookup to memory access
  - – What if PT lookup itself requires memory access? Head spinning…

Translation Buffer



- Functionality problem? Add indirection!
- Performance problem? Add cache!

- Address translation too slow?
  - Cache translations in **translation buffer (TB)**
    - Small cache: 16–64 entries, often fully assoc

+ Exploits temporal locality in PT accesses

+ OS handler only on TB miss

TLB Misses

- **TLB miss:** requested PTE not in TB, but in PT
  - Two ways of handling

- **1) OS routine**: reads PT, loads entry into TB (e.g., Alpha)
  - Privileged instructions in ISA for accessing TB directly
  - Latency: one or two memory accesses + OS call

- **2) Hardware FSM**: does same thing (e.g., IA-32)
  - Store PT root pointer in hardware register
  - Make PT root and 1st-level table pointers physical addresses
    - So FSM doesn't have to translate them

  \+ Latency: saves cost of OS call

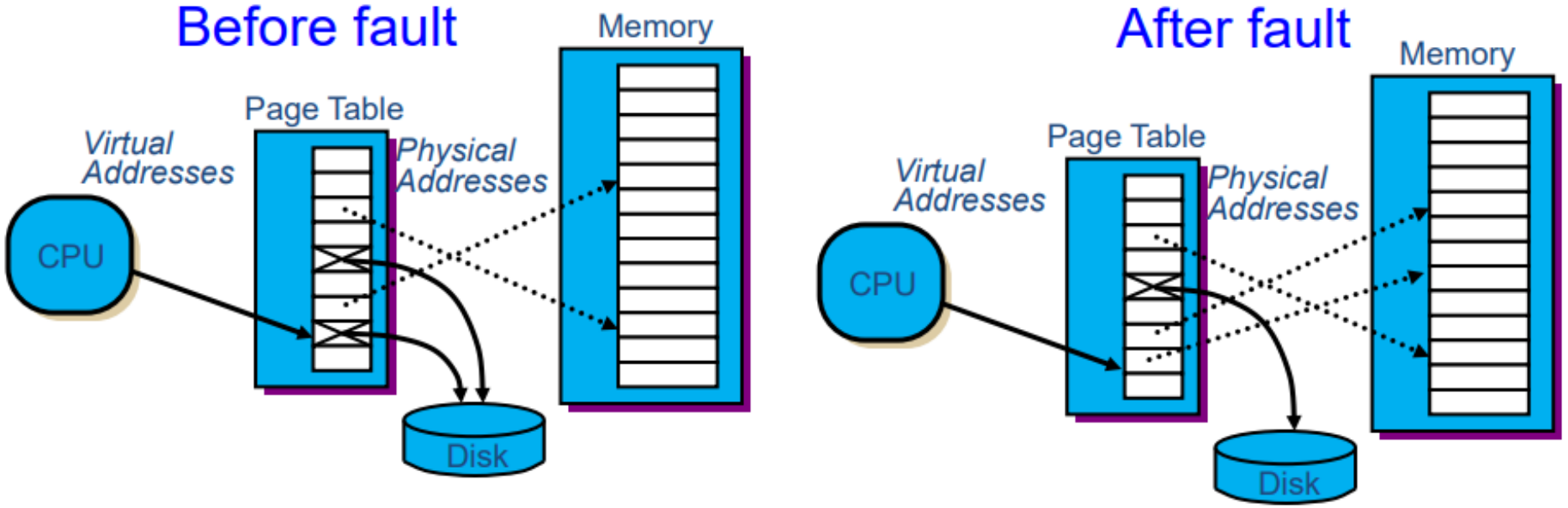
Page Faults

- **Page fault**: PTE not in TLB or in PT
  - Page is simply not in memory
  - Starts out as a TB miss, detected by OS handler/hardware FSM

- **OS routine**
  - OS software chooses a physical page to replace
    - **"Working set"**: more refined software version of LRU
      - Tries to see which pages are actively being used
      - Balances needs of all current running applications
    - If dirty, write to disk (like dirty cache block with writeback $)
  - Read missing page from disk (done by OS)
    - Takes so long (10ms), OS schedules another task
  - Treat like a normal TB miss from here

Handling Page Faults

- What if an object is on disk rather than in memory?
  - Page table entry indicates virtual address not in memory
  - OS exception handler invoked to move data from disk into memory
    - current process suspends, others can resume
    - OS has full control over placement, etc.



Selecting a Page Size

Selecting a Page Size

The most obvious architectural parameter is the page size. Choosing the page is a question of balancing forces that favor a larger page size versus those favoring a smaller size. The following favor a larger size:

- The size of the page table is inversely proportional to the page size; memory (or other resources used for the memory map) can therefore be saved by making the pages bigger.
- As mentioned in Section B.3, a larger page size can allow larger caches with fast cache hit times.
- Transferring larger pages to or from secondary storage, possibly over a network, is more efficient than transferring smaller pages.
- The number of TLB entries is restricted, so a larger page size means that more memory can be mapped efficiently, thereby reducing the number of TLB misses.

It is for this final reason that recent microprocessors have decided to support multiple page sizes; for some programs, TLB misses can be as significant on CPI as the cache misses.

Internal Fragmentation

The main motivation for a smaller page size is conserving storage. A small page size will result in less wasted storage when a contiguous region of virtual memory is not equal in size to a multiple of the page size. The term for this unused memory in a page is *internal fragmentation*. Assuming that each process has three primary segments (text, heap, and stack), the average wasted storage per process will be 1.5 times the page size. This amount is negligible for computers with hundreds of megabytes of memory and page sizes of 4–8 KiB. Of course, when the page sizes become very large (more than 32 KiB), storage (both main and secondary) could be wasted, as well as I/O bandwidth. A final concern is process start-up time; many processes are small, so a large page size would lengthen the time to invoke a process.

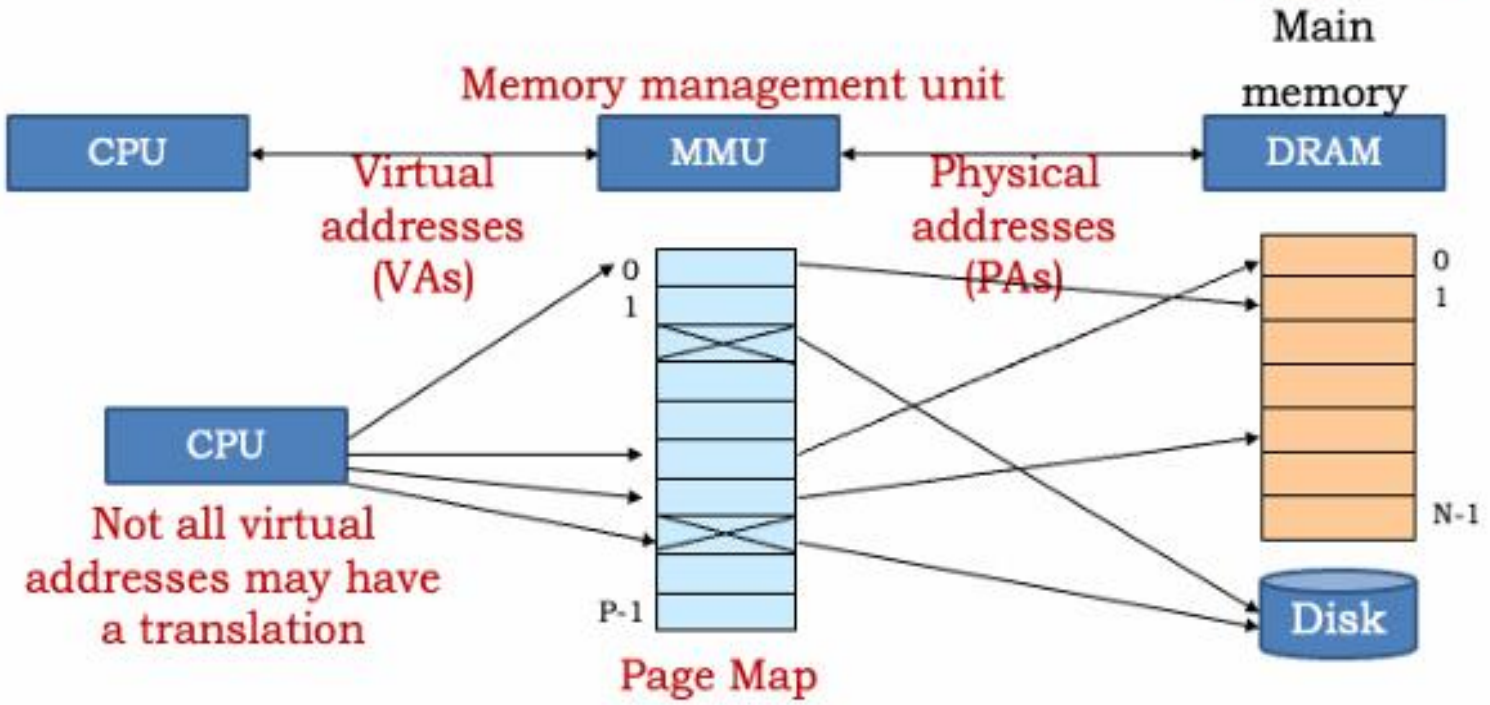
Another issue: Size of Page Tables

- Remember: For a 4 GiB address space, each process needs 4 GiB/4 KiB = 1 Million entries
  - Assuming 4 byte entries, 4 MiB per process
  - Per process! 256 concurrent processes consume 1 GiB just for the page table
  - “ps aux | wc -l” on my desktop tells me 261 processes are running…
- How do we reduce the size of page tables?
  - Larger page sizes? May waste memory capacity! (Fragmentation!)
  - Typical solution: Hierarchical page tables

Caches and Virtual Memory

Virtual Memory

- Two kinds of addresses:
  - CPU uses virtual addresses
  - Main memory uses physical addresses
- Hardware translates virtual addresses to physical addresses via an operating system (OS)-managed table, the page map



Where Can a Block be Placed in Main Memory?

Four Memory Hierarchy Questions Revisited

We are now ready to answer the four memory hierarchy questions for virtual memory.

Q1: Where Can a Block be Placed in Main Memory?

The miss penalty for virtual memory involves access to a rotating magnetic storage device and is therefore quite high. Given the choice of lower miss rates or a simpler placement algorithm, operating systems designers usually pick lower miss rates because of the exorbitant miss penalty. Thus, operating systems allow blocks to be placed anywhere in main memory. According to the terminology in Figure B.2 on page B-8, this strategy would be labeled fully associative.

How is a Block Found If it is in Main Memory?

Q2: How Is a Block Found If It Is in Main Memory?

Both paging and segmentation rely on a data structure that is indexed by the page or segment number. This data structure contains the physical address of the block. For segmentation, the offset is added to the segment's physical address to obtain the final physical address. For paging, the offset is simply concatenated to this physical page address (see Figure B.23).

This data structure, containing the physical page addresses, usually takes the form of a *page table*. Indexed by the virtual page number, the size of the table is the number of pages in the virtual address space. Given a 32-bit virtual address, 4 KiB pages, and 4 bytes per page table entry (PTE), the size of the page table would be $(2^{32}/2^{12}) \times 2^2 = 2^{22}$ or 4 MiB.

To reduce the size of this data structure, some computers apply a hashing function to the virtual address. The hash allows the data structure to be the length of the number of *physical* pages in main memory. This number could be much smaller than the number of virtual pages. Such a structure is called an *inverted page table*. Using the previous example, a 512 MiB physical memory would only need 1 MiB (8×512 MiB/4 KiB) for an inverted page table; the extra 4 bytes per page table entry are for the virtual address. The HP/Intel IA-64 covers both bases by offering both traditional pages tables *and* inverted page tables, leaving the choice of mechanism to the operating system programmer.

To reduce address translation time, computers use a cache dedicated to these address translations, called a *translation lookaside buffer*, or simply *translation buffer*, described in more detail shortly.

Which Block Should be Replaced on a Virtual Memory Miss?

Q3: Which Block Should be Replaced on a Virtual Memory Miss?

As mentioned earlier, the overriding operating system guideline is minimizing page faults. Consistent with this guideline, almost all operating systems try to replace the least recently used (LRU) block because if the past predicts the future, that is the one less likely to be needed.

To help the operating system estimate LRU, many processors provide a *use bit* or *reference bit*, which is logically set whenever a page is accessed. (To reduce work, it is actually set only on a translation buffer miss, which is described shortly.) The operating system periodically clears the use bits and later records them so it can determine which pages were touched during a particular time period. By keeping track in this way, the operating system can select a page that is among the least recently referenced.

What Happens on a Write in Virtual Memory?

Q4: What Happens on a Write?

The level below main memory contains rotating magnetic disks that take millions of clock cycles to access. Because of the great discrepancy in access time, no one has yet built a virtual memory operating system that writes through main memory to disk on every store by the processor. (This remark should not be interpreted as an opportunity to become famous by being the first to build one!) Thus, the write strategy is always write-back.

Because the cost of an unnecessary access to the next-lower level is so high, virtual memory systems usually include a dirty bit. It allows blocks to be written to disk only if they have been altered since being read from the disk.

Summary of Virtual Memory and Caches

Summary of Virtual Memory and Caches

With virtual memory, TLBs, first-level caches, and second-level caches all mapping portions of the virtual and physical address space, it can get confusing what bits go where. Figure B.25 gives a hypothetical example going from a 64-bit virtual address to a 41-bit physical address with two levels of cache. This L1 cache is virtually indexed, and physically tagged because both the cache size and the page size are 8 KiB. The L2 cache is 4 MiB. The block size for both is 64 bytes.

Memory Hierarchy from VA to L2 Cache

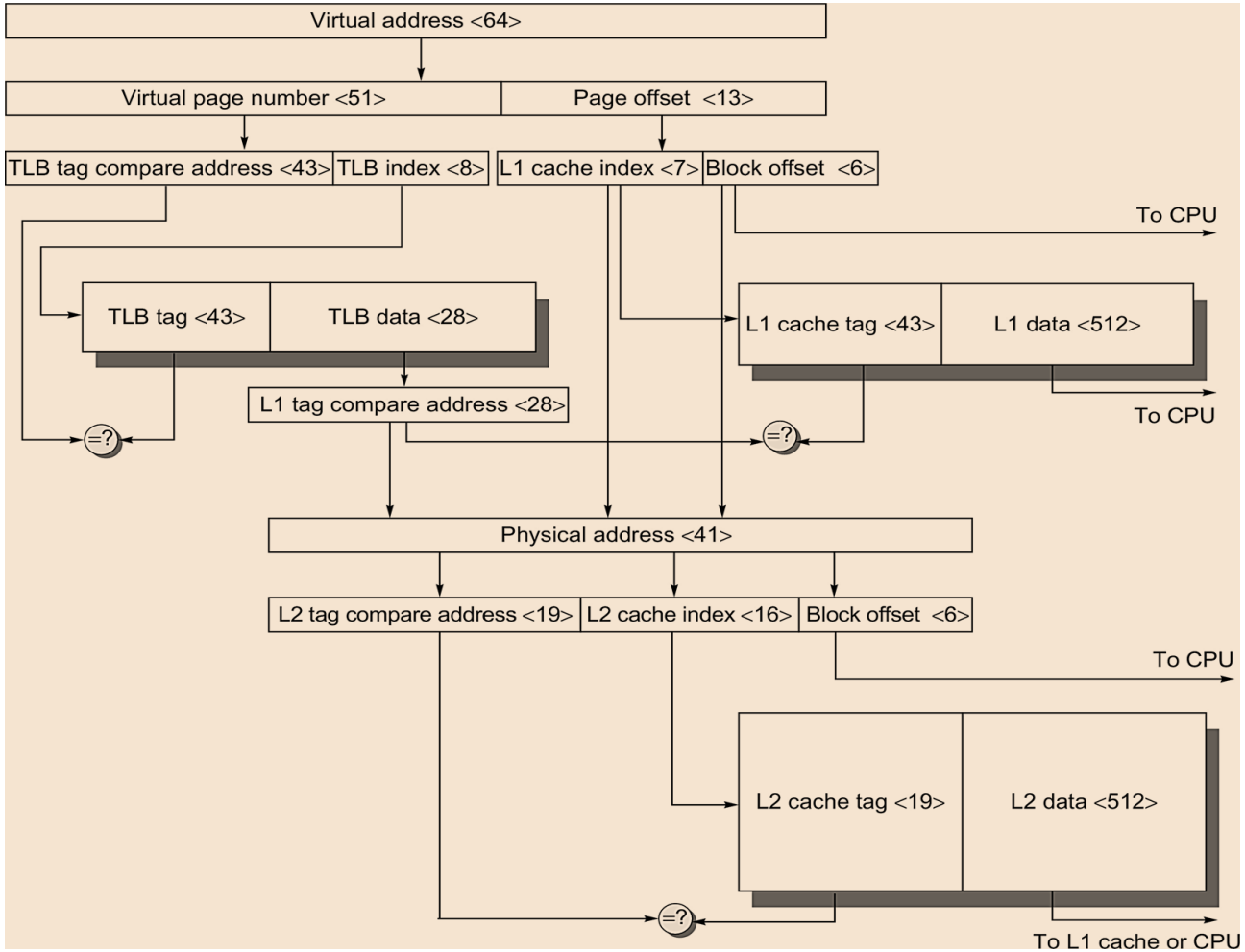


Figure B.25 The overall picture of a hypothetical memory hierarchy going from virtual address to L2 cache access. The page size is 8 KiB. The TLB is direct mapped with 256 entries. The L1 cache is a direct-mapped 8 KiB, and the L2 cache is a direct-mapped 4 MiB. Both use 64-byte blocks. The virtual address is 64 bits and the physical address is 41 bits. The primary difference between this simple figure and a real cache is replication of pieces of this figure.

Explanation VA to L2 Cache

First, the 64-bit virtual address is logically divided into a virtual page number and page offset. The former is sent to the TLB to be translated into a physical address, and the high bit of the latter is sent to the L1 cache to act as an index. If the TLB match is a hit, then the physical page number is sent to the L1 cache tag to check for a match. If it matches, it's an L1 cache hit. The block offset then selects the word for the processor.

If the L1 cache check results in a miss, the physical address is then used to try the L2 cache. The middle portion of the physical address is used as an index to the 4 MiB L2 cache. The resulting L2 cache tag is compared with the upper part of the physical address to check for a match. If it matches, we have an L2 cache hit, and the data are sent to the processor, which uses the block offset to select the desired word. On an L2 miss, the physical address is then used to get the block from memory.

Although this is a simple example, the major difference between this drawing and a real cache is replication. First, there is only one L1 cache. When there are two L1 caches, the top half of the diagram is duplicated. Note that this would lead to two TLBs, which is typical. Hence, one cache and TLB is for instructions, driven from the PC, and one cache and TLB is for data, driven from the effective address.

The second simplification is that all the caches and TLBs are direct mapped. If any were n-way set associative, then we would replicate each set of tag memory, comparators, and data memory n times and connect data memories with an n:1 multiplexor to select a hit. Of course, if the total cache size remained the same, the cache index would also shrink by $\log 2n$ bits according to the formula in Figure B.7 on page B-22.

Differences Between Cache and Virtual Memory

There are further differences between caches and virtual memory beyond those quantitative ones mentioned in Figure B.20:

- Replacement on cache misses is primarily controlled by hardware, while virtual memory replacement is primarily controlled by the operating system. The longer miss penalty means it's more important to make a good decision, so the operating system can be involved and take time deciding what to replace.
- The size of the processor address determines the size of virtual memory, but the cache size is independent of the processor address size.
- In addition to acting as the lower-level backing store for main memory in the hierarchy, secondary storage is also used for the file system. In fact, the file system occupies most of secondary storage. It is not usually in the address space.

Typical Parameters for VM and Caches

Parameter	First-level cache	Virtual memory
Block (page) size	16–128 bytes	4096–65,536 bytes
Hit time	1–3 clock cycles	100–200 clock cycles
Miss penalty	8–200 clock cycles	1,000,000–10,000,000 clock cycles
(access time)	(6–160 clock cycles)	(800,000–8,000,000 clock cycles)
(transfer time)	(2–40 clock cycles)	(200,000–2,000,000 clock cycles)
Miss rate	0.1%–10%	0.00001%–0.001%
Address mapping	25–45-bit physical address to 14–20-bit cache address	32–64-bit virtual address to 25–45-bit physical address

Figure B.20 Typical ranges of parameters for caches and virtual memory. Virtual memory parameters represent increases of 10–1,000,000 times over cache parameters. Usually, first-level caches contain at most 1 MiB of data, whereas physical memory contains 256 MiB to 1 TB.

- https://www.intel.com/content/www/us/en/docs/programmable/683567/24-3/system-mmu-block-diagram-falconmesa.html